สัปดาห์ที่ 2 · 05_DAG_Planner

DAG Planner Skill

แตกงานเป็นบล็อก · แต่ละบล็อกใช้สกิล · DAG + Gantt · ต่อยอดจาก base skills

สกิลตัวที่ 4 ต่อยอดจาก base skills **yolo · cyolo · agent-team** ให้ AI วางแผนว่าใครทำบล็อกไหน เมื่อไร · คาบนี้วางแผนอย่างเดียว ยังไม่รัน

ส่วนที่ 1 จาก 6

ปัญหาและทฤษฎี

บท 04 ให้ **ทีม** · บทนี้ให้ **แผน** ว่าใครทำอะไร ก่อน-หลัง เมื่อไร

1 · ปัญหา

ก่อนเขียน graph.json ต้องตอบให้ได้

คำถาม	ถ้าตอบไม่ได้
งานมีกี่ชิ้น ชิ้นไหนต้องรอชิ้นไหน	สั่งงานผิดลำดับ
แต่ละชิ้นควรใช้สกิลไหน	ใช้บัญชีแพงกับงานง่าย หรือไม่มีใครตรวจงานสำคัญ
ชิ้นไหนให้บัญชีไหน เมื่อไร	บัญชีโควตาน้อยรับงานใหญ่จนค้าง
ทั้งโปรเจกต์จะเสร็จเมื่อไร	ไม่รู้ว่าเพิ่มบัญชีแล้วเร็วขึ้นไหม

งานใหญ่ สั่งทีเดียวไม่ไหว

จึงแตกเป็นบล็อกงานชิ้นเล็ก

ปัญหา

- ✕ ไม่รู้จะเริ่มตรงไหน
- ✕ AI ตัวเดียวทำไม่จบ
- ✕ พลาดจุดเดียวต้องเริ่มใหม่

แตกงาน

บล็อกงาน

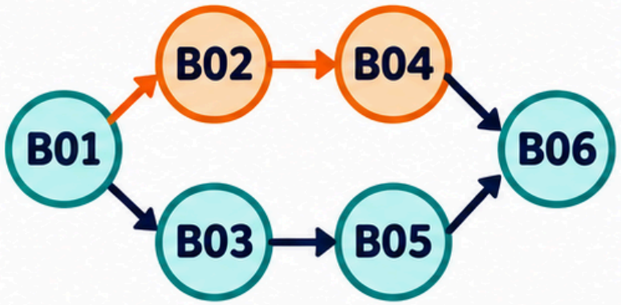
B01 yolo	B02 cyolo
B03 image	B04 review
B05 yolo	B06 cyolo

บล็อกงาน = งานชิ้นเล็ก 1 ชิ้น ที่บอกว่าใช้สกิลอะไรทำ ได้ไฟล์อะไร และเสร็จแล้วตรวจอย่างไร

จากบล็อกงาน สู่แผนที่รันได้จริง

DAG

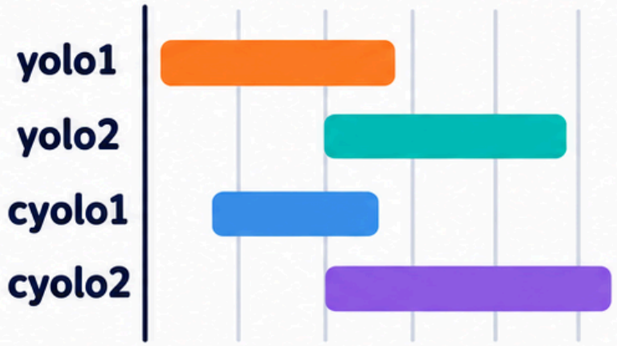
Directed Acyclic Graph



แผนผังว่างานไหนต้องรองานไหน
ลูกศร = ต้องรอ • ห้ามวนกลับ

Gantt

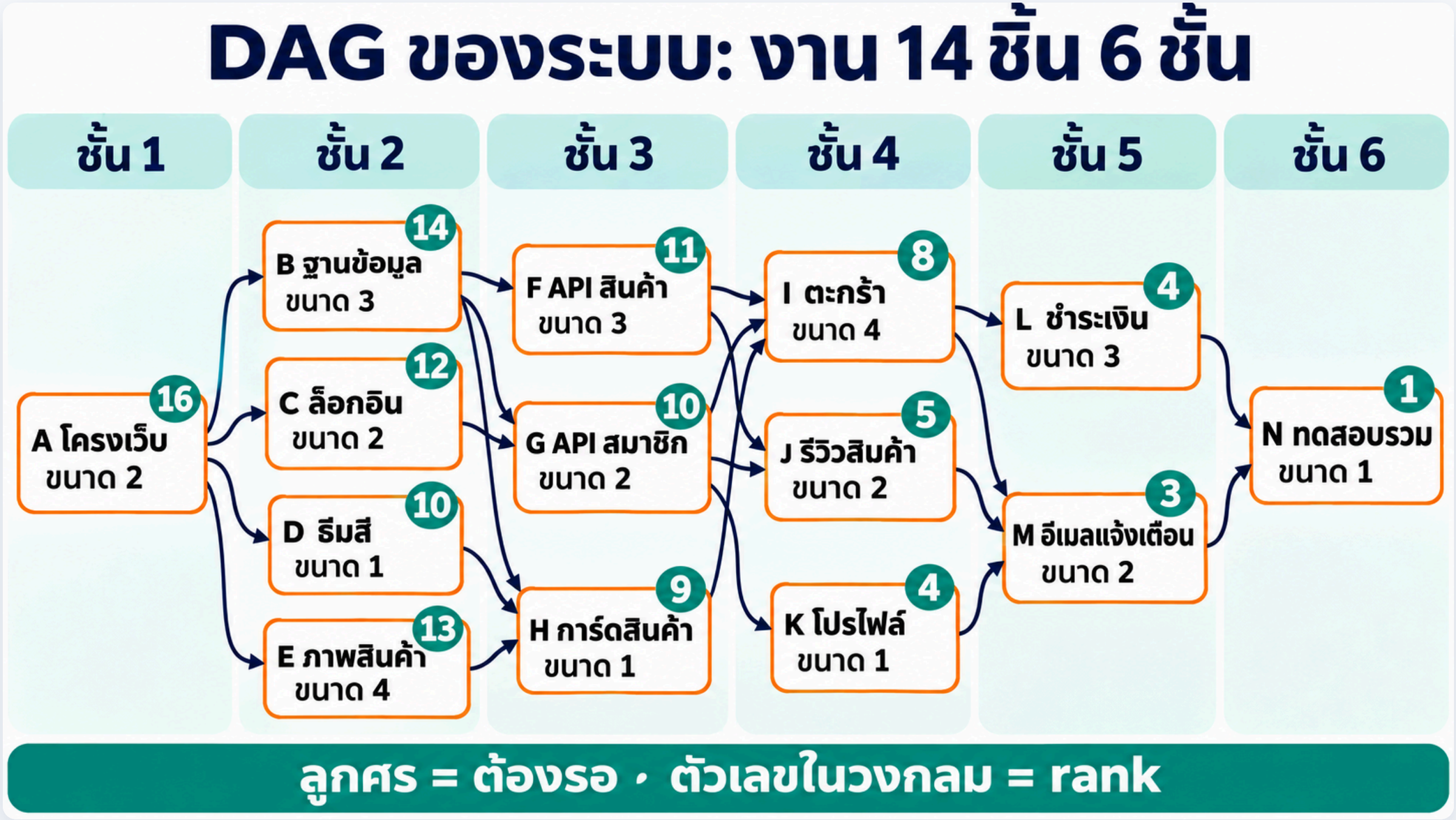
แกนต์ชาร์ต



ตารางเวลาว่าใครทำงานไหน เมื่อไร
แถว = บัญชี • แท่ง = งาน

รันจริง

ทีมเอเจนต์ทำตามแผน
แล้วเทียบแผนกับเวลาจริง


DAG ของระบบ: งาน 14 ชิ้น 6 ชั้น
ชั้น 1
ชั้น 2
ชั้น 3
ชั้น 4
ชั้น 5
ชั้น 6
A โครงเว็บ
ขนาด 2
16
B ฐานข้อมูล
ขนาด 3
14
C ล็อกอิน
ขนาด 2
12
D ธีมสี
ขนาด 1
10
E ภาพสินค้า
ขนาด 4
13
F API สินค้า
ขนาด 3
11
G API สมาชิก
ขนาด 2
10
H การ์ดสินค้า
ขนาด 1
9
I ตะกร้า
ขนาด 4
8
J รีวิวสินค้า
ขนาด 2
5
K โปรไฟล์
ขนาด 1
4
L ชำระเงิน
ขนาด 3
4
M อีเมลแจ้งเตือน
ขนาด 2
3
N ทดสอบรวม
ขนาด 1
1
ลูกศร = ต้องรอ · ตัวเลขในวงกลม = rank

คิด rank ย้อนจากชั้นสุดท้าย

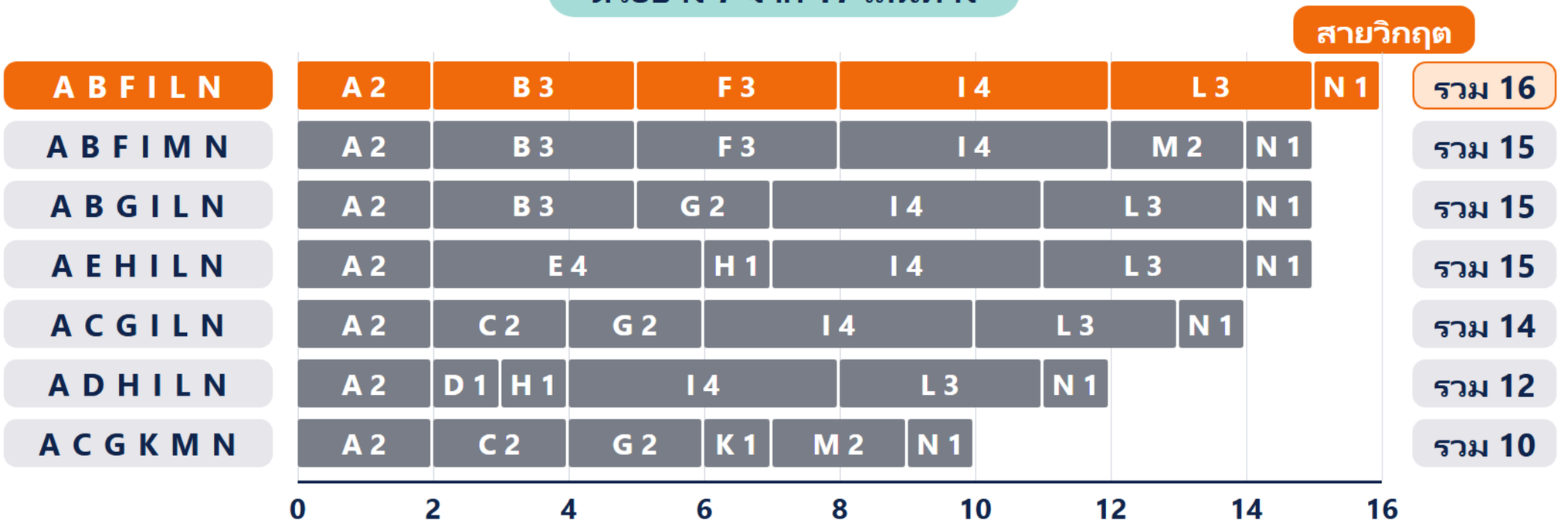
ชั้น 6	N = 1			
ชั้น 5	L = 3 + 1 = 4	M = 2 + 1 = 3		
ชั้น 4	I = 4 + max(4, 3) = 8	J = 2 + 3 = 5	K = 1 + 3 = 4	
ชั้น 3	F = 3 + max(8, 5) = 11	G = 2 + max(8, 5, 4) = 10	H = 1 + 8 = 9	
ชั้น 2	B = 3 + max(11, 10, 9) = 14	C = 2 + 10 = 12	D = 1 + 9 = 10	E = 4 + 9 = 13
ชั้น 1	A = 2 + max(14, 12, 10, 13) = 16			

คิดจากบนลงล่าง

rank = ขนาดของงาน + rank ที่มากที่สุดของงานที่รอมันอยู่

สายวิกฤต = เส้นทางที่ยาวที่สุด

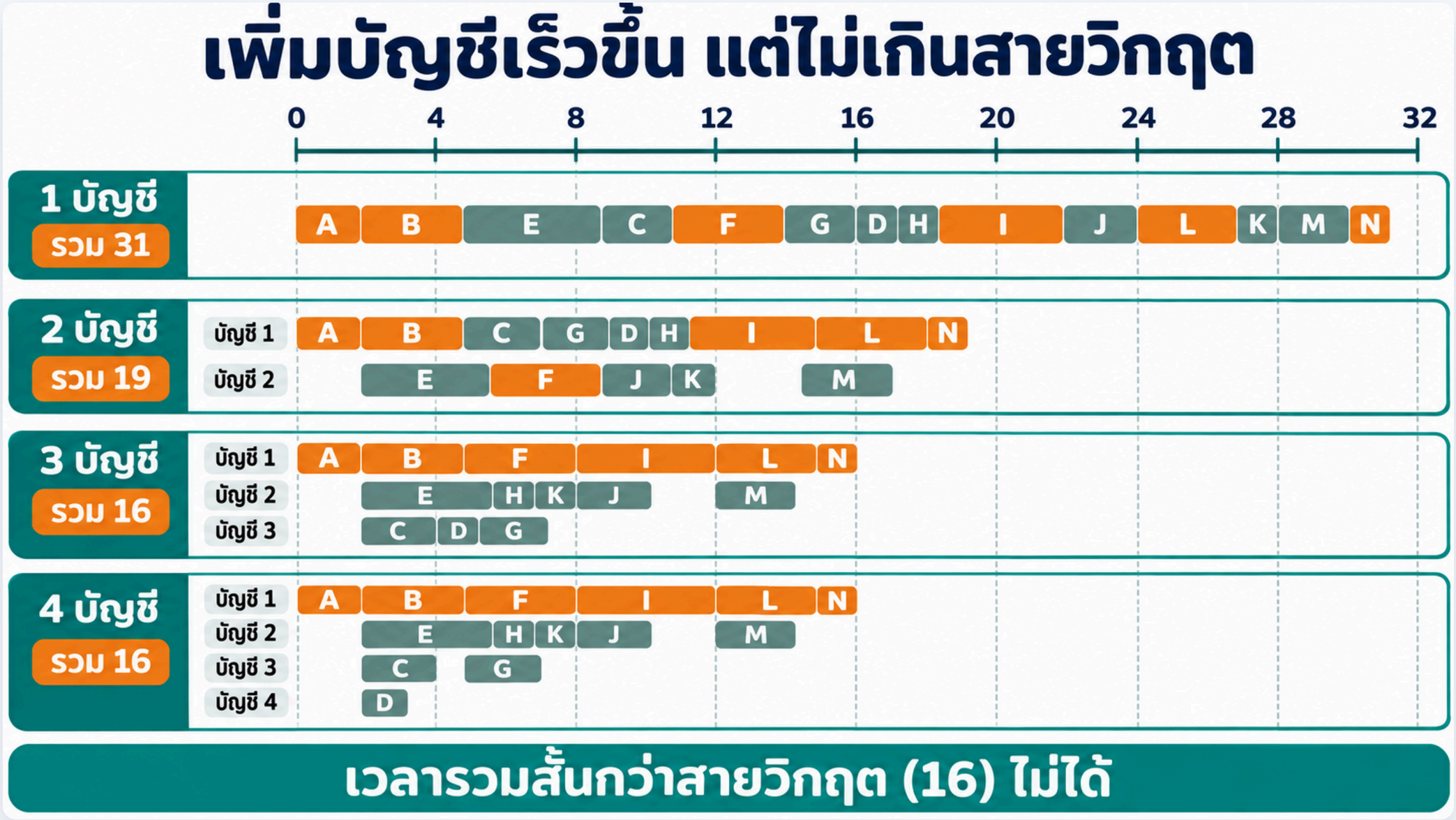
ตัวอย่าง 7 จาก 17 เส้นทาง

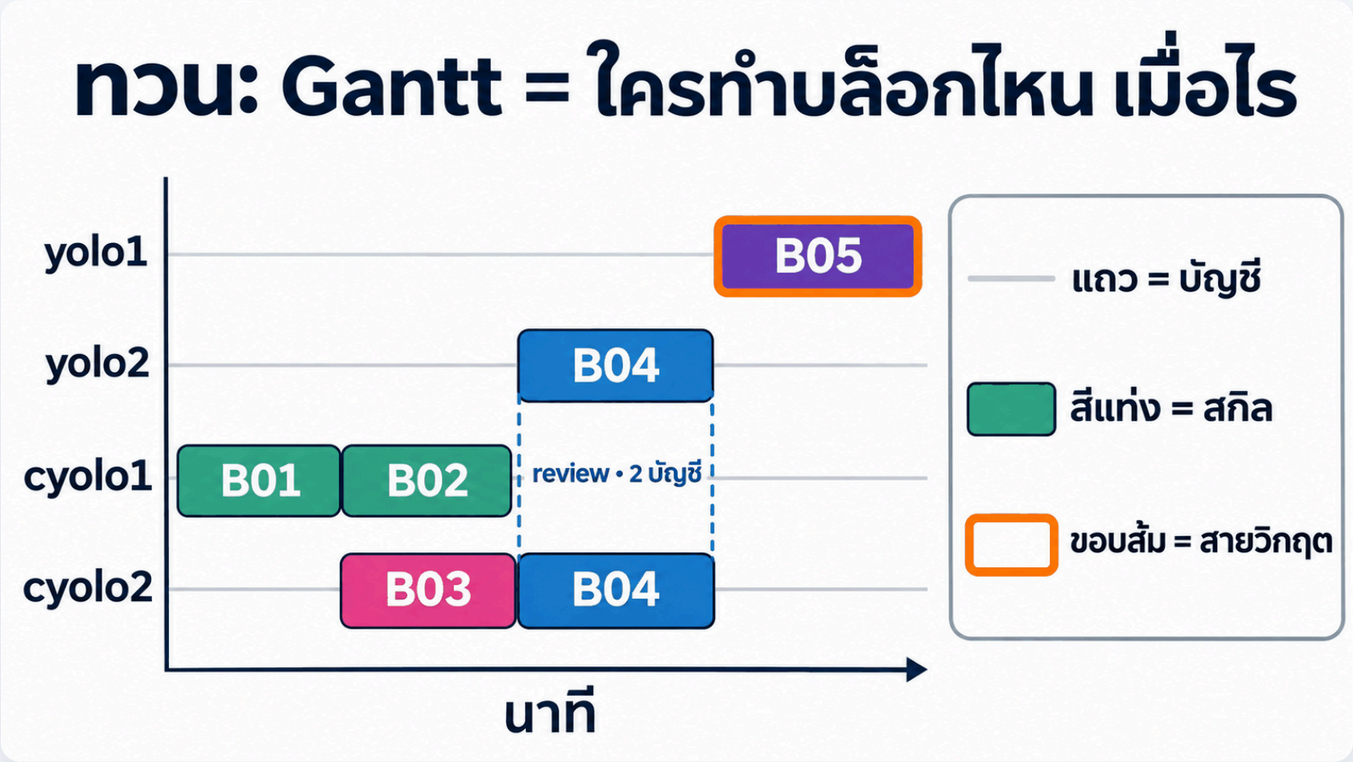


F ช้าลง 1
ทั้งโปรเจกต์ช้าเป็น 17

E ช้าลง 2
ทั้งโปรเจกต์ช้าเป็น 17

D ช้าลง 4
ทั้งโปรเจกต์ยังเป็น 16


เพิ่มบัญชีเร็วขึ้น แต่ไม่เกินสายวิกฤต
0
4
8
12
16
20
24
28
32
1 บัญชี
รวม 31
A
B
E
C
F
G
D
H
I
J
L
K
M
N
2 บัญชี
รวม 19
บัญชี 1
บัญชี 2
3 บัญชี
รวม 16
บัญชี 1
บัญชี 2
บัญชี 3
4 บัญชี
รวม 16
บัญชี 1
บัญชี 2
บัญชี 3
บัญชี 4
เวลารวมสั้นกว่าสายวิกฤต (16) ไม่ได้



- แถว = บัญชี · แท่ง = บล็อก · **สีแท่ง = สกิล**
- บล็อกหลายบัญชีมีหลายแถว
- เวลารวมสั้นกว่าสายวิกฤตไม่ได้
- ใครทำบล็อกไหน = **KC-QUEST** (บท 06)

ส่วนที่ 2 จาก 6

บล็อกงาน = การ์ดที่บอกว่าใช้สกิลอะไร

แต่ละบล็อกระบุ **skill** + **difficulty** + ไฟล์ผล + เกณฑ์เสร็จ

ใบสั่ง = บล็อก · สถานี = สกิล

บางจานต้องมีคนชิมก่อนเสิร์ฟ (review) · บางจานให้หลายคนเสนอแล้วเลือก (best_of)

แต่ละบล็อกเลือกสกิลที่ใช้ทำ

yolo

งานที่ต้องคิดยาว

1 บัญชี

cyolo

งานตรงไปตรงมา

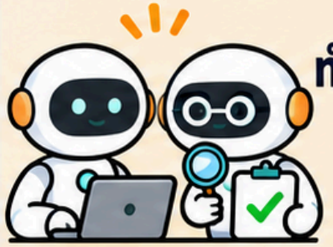
1 บัญชี

image

สร้างภาพ

1 บัญชี

review



ทำ แล้วให้อีกฝั่งตรวจ

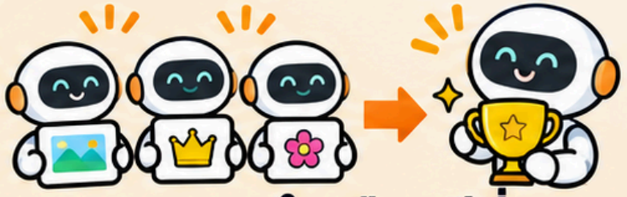
2 บัญชี

debate

สองฝั่งเถียง มีผู้ตัดสิน

3 บัญชี

best_of



หลายคนทำ เลือกดีที่สุด

N+1 บัญชี

งานยิ่งสำคัญ ยิ่งใช้หลายบัญชี แต่เปลืองโทเคนขึ้น

2 · สกิลของบล็อก → สิ่งที่ทีมทำจริง

skill	ใช้เมื่อ	agent-team ทำจริง	บัญชี
🧠 `yolo`	เขียนโค้ด/เอกสาร งานคิดยาว	งานเดี่ยวฝั่ง Claude	1
⚙️ `cyolo`	โค้ด/ข้อมูลตรงไปตรงมา	งานเดี่ยวฝั่ง Codex	1
🖼️ `image`	สร้างภาพ	งานภาพ (Codex เสมอ)	1
🔍 `review`	งานสำคัญต้องมีคนตรวจ	ผู้ทำ ↔ ผู้ตรวจคนละ CLI	2
⚖️ `debate`	ตัดสินใจที่มีถูก/ผิด	Claude vs Codex + ผู้ตัดสิน	3
🏆 `best_of`	งานสร้างสรรค์	หลายคนทำ + ผู้ตัดสิน	N+1

review · debate · best_of ใช้หลายบัญชี → ใช้เฉพาะบล็อกที่คุ้ม

บล็อกงาน 1 บล็อกบอกอะไรบ้าง

AI กรอก

I ตะกร้า

ขนาด	L
ต้องรอ	F, G, H
สกิล	yolo
ความยาก	3
ไฟล์ผล	cart.js
เกณฑ์เสร็จ	รีเฟรชแล้วตะกร้ายังอยู่

สกิลคำนวณให้

rank

สายวิกฤต

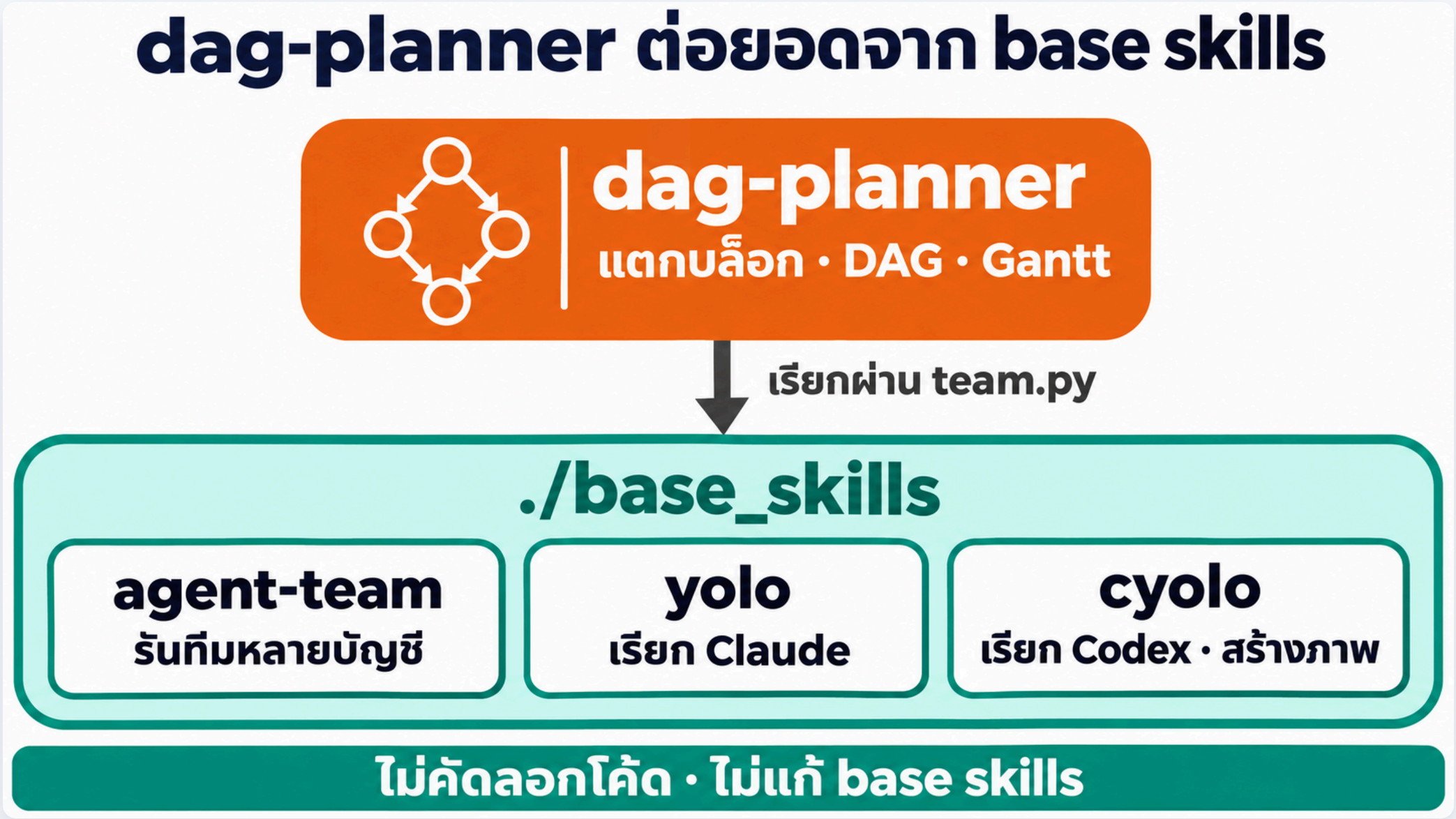
บัญชีที่ทำ

AI บอกว่าต้องทำอะไร · สกิลคำนวณว่าใครทำเมื่อไร

ส่วนที่ 3 จาก 6

สกิล dag-planner

AI = **สมอง** แตกบล็อก · dagplan.py = **มือ** วาด DAG/Gantt · base skills = **ทีม** ที่รับแผนไปรัน


dag-planner ต่อยอดจาก base skills
dag-planner
แตกบล็อก · DAG · Gantt
เรียกผ่าน team.py
./base_skills
agent-team
รันทีมหลายบัญชี
yolo
เรียก Claude
cyolo
เรียก Codex · สร้างภาพ
ไม่คัดลอกโค้ด · ไม่แก้ base skills

3 · base skills ในโฟลเดอร์ ./base_skills

base skill	ทำอะไร	dag-planner ใช้ตรงไหน
`agent-team`	รันทีมหลายบัญชีตาม graph.json · ตรวจความพร้อมและโควตา	`team.py check` · `quota` ตอน plan · `team.py run` ตอนรันจริง
`yolo`	เรียก Claude 1 บัญชี	บล็อก `yolo` และฝั่ง Claude ของ review · debate · best_of
`cyolo`	เรียก Codex 1 บัญชี · สร้างภาพ	บล็อก `cyolo` · `image` และฝั่ง Codex ของ review · debate · best_of

กติกา 2 ข้อ: **ไม่คัดลอกโค้ด**ของ base skill (เรียกผ่าน team.py เท่านั้น) และ**ไม่แก้ไฟล์**ใน ./base_skills · base skill เปลี่ยนรุ่น → สร้าง dag-planner ใหม่จากสเปกเดิม

สกิล dag-planner ทำงานอย่างไร

AI คิด ใช้โทเคน

1 โจทย์

ทำ เล่าสิ่งที่อยากได้

ได้ โจทย์ 1 ข้อ

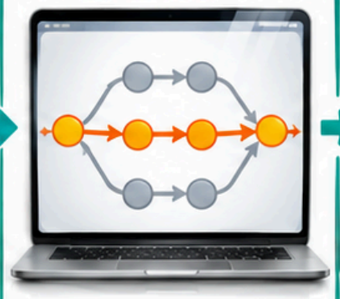
2 แตกบล็อก

ทำ เลือกสกิลและความยาก

ได้ blocks.json

สคริปต์คำนวณ ไม่ใช้โทเคน

3 DAG



ทำ คำสั่ง dag คิด rank

ได้ dag.png

4 Gantt แผน

ทำ คำสั่ง plan แจกบัญชี

ได้ gantt-plan.png

ทีมเอเจนต์ทำ ใช้โทเคน

5 รันจริง

ทำ คำสั่ง export + run

ได้ ไฟล์งานใน app/

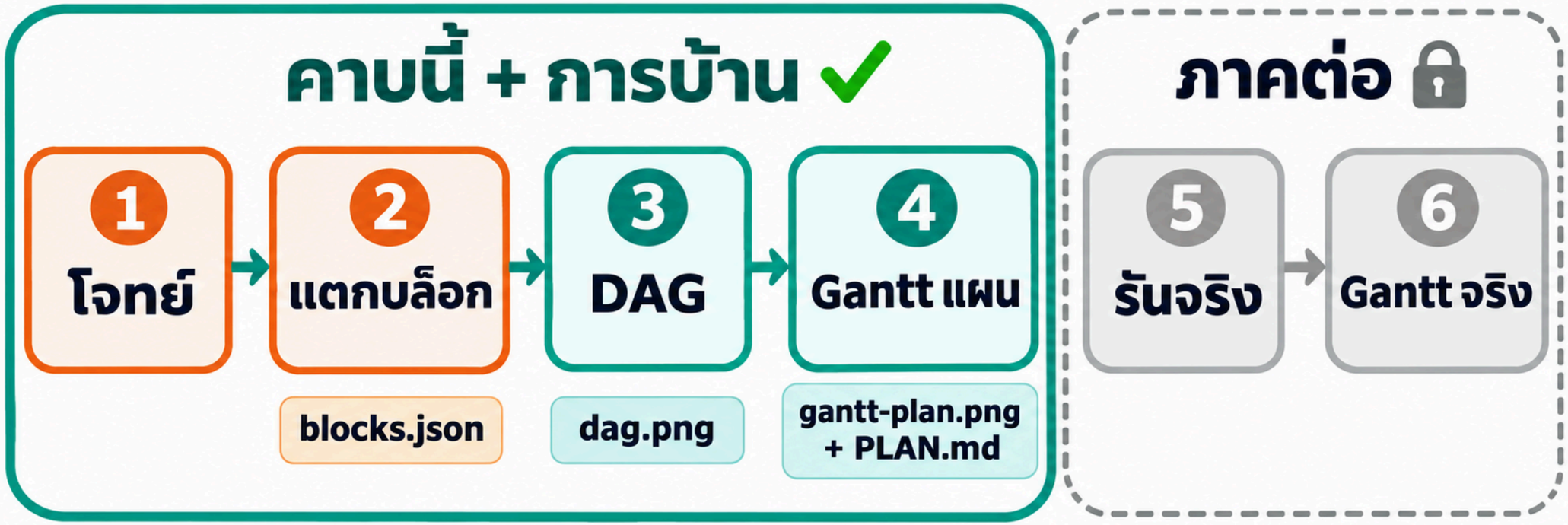
6 Gantt จริง

ทำ คำสั่ง report เทียบแผน

ได้ REPORT.md

calib.json เก็บตัวปรับเวลาของแต่ละสกิล ให้แผนรอบหน้าแม่นขึ้น

คาบนี้วางแผนอย่างเดียว ยังไม่รัน

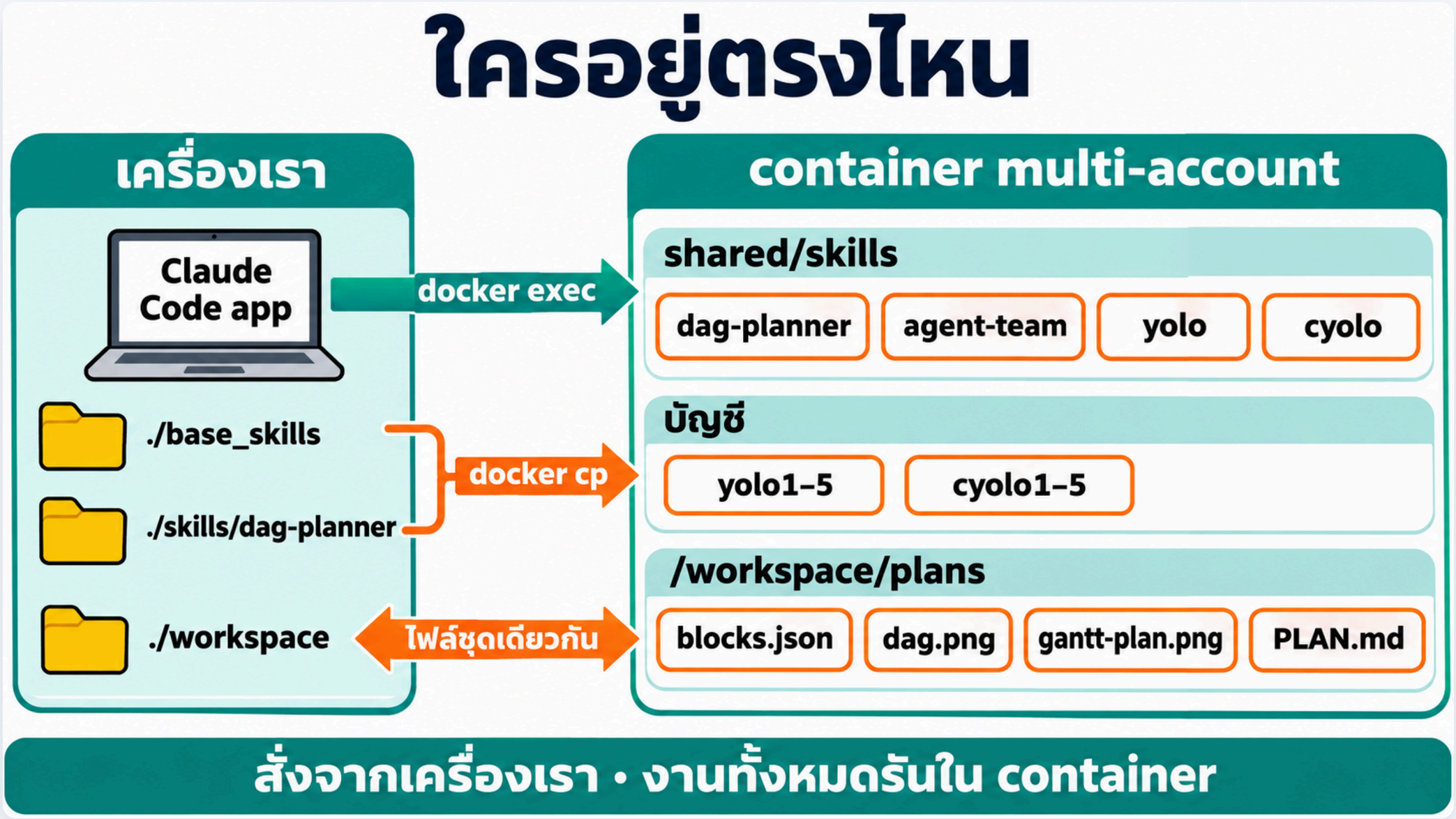


วางแผนไม่ใช้โทเคนของทีม

3 · ใครทำอะไร

ใคร	ทำอะไร	โทเคน
AI = สมอง (Claude Code app)	อ่านโจทย์ แตกบล็อก เลือกสกิลและความยาก	ใช้
dagplan.py = มือ	ตรวจกราฟ · rank · DAG · KC-QUEST แจกบัญชี · Gantt · export · report	**ไม่ใช้**
agent-team = ทีม (base skill)	รันแต่ละบล็อกด้วยสกิลที่ระบุ บนบัญชีจริง	ใช้

คำสั่ง	ผล
`doctor`	เครื่องพร้อมไหม · พบ base skills ที่ไหน
`dag`	ตรวจบล็อก + rank + dag.png
`plan`	KC-QUEST → gantt-plan.png · PLAN.md
`export` · `run` ภาคต่อ	graph.json → agent-team รันจริง
`report` ภาคต่อ	gantt-actual · compare · REPORT.md · calib.json


ใครอยู่ตรงไหน
เครื่องเรา
Claude Code app
./base_skills
./skills/dag-planner
./workspace
docker exec
docker cp
ไฟล์ชุดเดียวกัน
container multi-account
shared/skills
dag-planner
agent-team
yolo
cyolo
บัญชี
yolo1–5
cyolo1–5
/workspace/plans
blocks.json
dag.png
gantt-plan.png
PLAN.md
สั่งจากเครื่องเรา • งานทั้งหมดรันใน container

ภาพจำลอง

k

Gantt แผน = ETA ของแอปนำทาง

report เทียบกับของจริง แล้วคำนวณ k = จริง ÷ แผน แยกตามสกิล เก็บใน calib.json

3 · แผน vs ของจริง

k_{skill} = เวลาจริง ÷ เวลาแผน

เวลาต่าง → เรียนรู้ k

report เก็บ k แยกตามสกิลใน calib.json · plan ครั้งถัดไปคูณเวลาด้วย k (KC-QUEST ขั้น ⑥ เรียนรู้)

บัญชีต่าง → failover ยังทำงาน

export ส่งเป็นฝั่ง yolo/cyolo ไม่ผูกบัญชี agent-team เลือกบัญชีว่างเอง report แสดงส่วนต่าง

ส่วนที่ 4 จาก 6

ลงมือ 3 ขั้น

สร้างสกิลจากสเปก + ./base_skills → ติดตั้ง → วางแผนเหมียวช็อป **(ยังไม่รัน)**

4 · ภาพรวมขั้นตอนการทดลอง

ขั้น	โทเคนของทีม
1 สร้างสกิลจากสเปก + ./base_skills	ไม่ใช้
2 ติดตั้งด้วยการคัดลอกโฟลเดอร์	ไม่ใช้
3 วางแผนเหมียวช็อป (ยังไม่รัน)	ไม่ใช้
การบ้าน วางแผน 10 โจทย์	ไม่ใช้
ภาคต่อ · รันแผนจริงและ what-if	**ใช้**

```
05_DAG_Planner/
├── base_skills/              ← agent-team · yolo · cyolo (input 1 · ห้ามแก้)
├── prompt_dag_skill_spec.md  ← สเปก (input 2)
├── tools/                    ← ตัววางแผนเดิม (input 3)
├── homework/                 ← โจทย์การบ้าน 10 ข้อ + ตัวอย่างข้อ 1
└── _solution/                ← สกิล · TEST-REPORT · เฉลยการบ้าน · แผน meow-shop
```

ต้องมี container multi-account · ล็อกอินอย่างน้อย 2 บัญชี Claude + 2 บัญชี Codex · ไม่จำเป็นต้องทำบท 04 มาก่อน เพราะ base skills อยู่ใน ./base_skills แล้ว

1

ให้ Claude สร้างสกิลจากสเปก + base skills

input: **./base_skills** + **prompt_dag_skill_spec.md** + **tools/** (ตัววางแผนเดิม) · ทดสอบโดยไม่ใช้โทเคนของทีม

ขั้นที่ 1 · input 3 อย่าง

input	บทบาท
`base_skills/`	base skills 3 ตัว Claude อ่าน SKILL.md · references · scripts/team.py ของ agent-team เพื่อรู้รูปแบบ graph.json และผลของ check · quota จริง
`prompt_dag_skill_spec.md`	สเปกของสกิล: ฟิลด์ของบล็อก · ตารางสกิล → node · คำสั่ง 6 คำสั่ง · ภาพ · ตัวปรับ k · การติดตั้ง · เกณฑ์ทดสอบ 8 ข้อ
`tools/`	planner.py + orch.py ตัววางแผนชุดเดิม ใช้เป็นแหล่งความรู้ ห้ามคัดลอกทั้งไฟล์ ดึงเฉพาะส่วน KC-QUEST

> ให้ base skills เป็นไฟล์ในโฟลเดอร์งาน: Claude อ่านได้ตรง ๆ ไม่ต้องเข้าไปค้นใน container และผู้เรียนเห็นว่าสกิลใหม่ต่อยอดจากอะไร

ขั้นที่ 1 · พรอมต์ (คัดลอกทั้งกรอบ)

```
ช่วยสร้างสกิล dag-planner ที่ทำให้ AI วางแผนงานเป็น DAG + Gantt โดยแต่ละบล็อกงานใช้ base skill ใน ./base_skills แล้วส่งแผนให้ทีมเอเจนต์รันจริงได้
ทำในโฟลเดอร์นี้ ใช้คำสั่ง docker ได้ แต่ห้ามลบ หยุด หรือสร้าง container / volume ใหม่ ห้ามแตะ credential และห้ามล็อกอินแทนผม

วัตถุดิบ
- ./base_skills = base skills 3 ตัว (agent-team · yolo · cyolo) อ่าน SKILL.md references และ scripts ของแต่ละตัวก่อนใช้ ห้ามเดา ห้ามแก้ไฟล์ในโฟลเดอร์นี้
- ./prompt_dag_skill_spec.md = สเปกของสกิล ทุกข้อต้องเป็นจริง
- ./tools/planner.py และ ./tools/orch.py = ตัววางแผนชุดเดิม (DAG · Gantt · KC-QUEST) ใช้เป็นแหล่งความรู้ ห้ามคัดลอกทั้งไฟล์
- container multi-account ที่กำลังรัน = ที่ทดสอบจริง

สร้างไว้ใน ./skills/dag-planner (นอก container = ต้นฉบับที่เก็บสำรอง)
ทดสอบ: คัดลอก base skills (ถ้าใน container ยังไม่มีหรือไม่ตรงกับ ./base_skills) และ dag-planner ไปวางที่ shared ของทั้งสองฝั่งใน container แล้วทดสอบทุกข้อในหัวข้อ "ทดสอบ" ของสเปก
ห้ามรันทีมจริงด้วยโทเคน ทดสอบการรันด้วยตัวจำลองของ agent-team เท่านั้น ข้อที่ต้องใช้บัญชีที่ไม่พร้อม = SKIP
เขียน ./TEST-REPORT.md (นอกโฟลเดอร์ skills) แล้วสรุปเป็นตารางสั้น ๆ: ผลทดสอบ ไฟล์ที่สร้าง และสิ่งที่ยังไม่ได้ทดสอบ แล้วหยุด
```

ขั้นที่ 1 · ผลจริง 10 พ.ย. 2026 · ไม่ได้รันทีมจริงด้วยโทเคน

PASS 8 / 8

✻ Claude · ขั้นที่ 1 · สร้างสกิล dag-planner จากสเปก + ./base_skills · ผลทดสอบจริง 10 พ.ย. 2026 (จัดหน้าใหม่ให้อ่านง่าย)

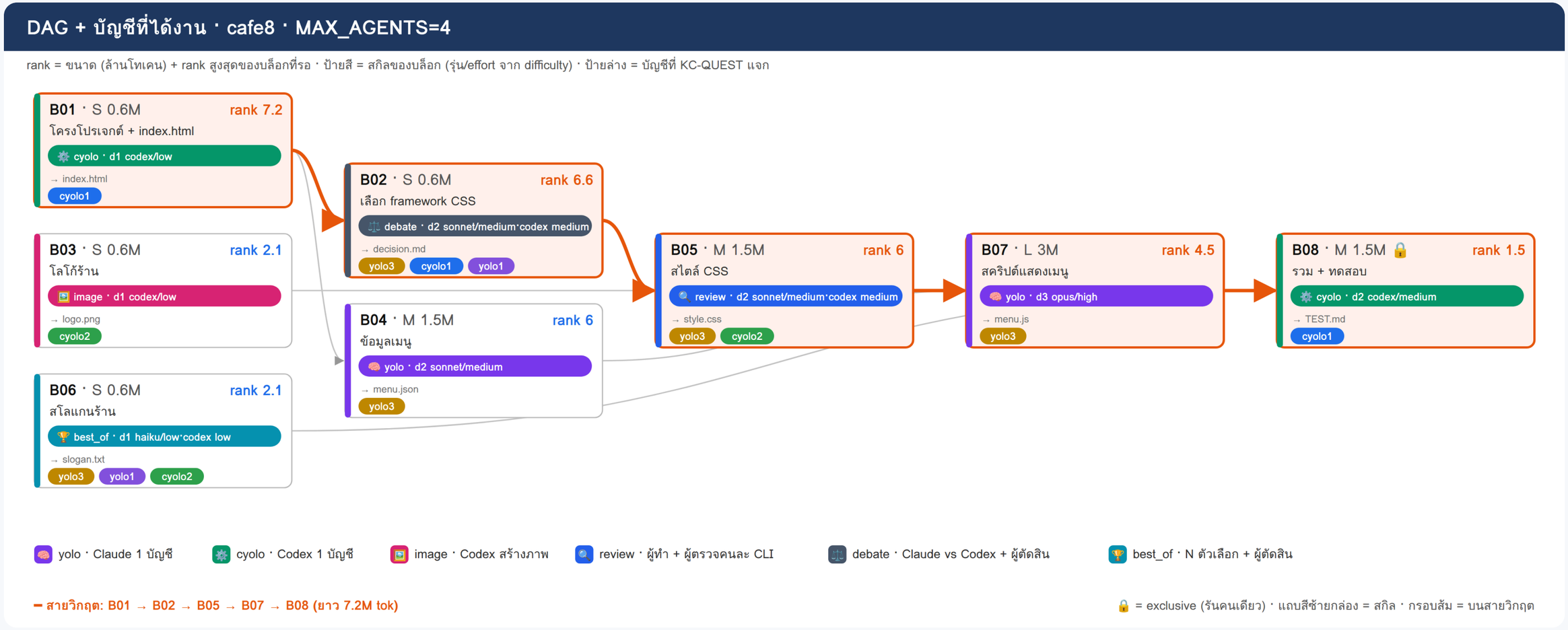
สร้าง `skills/dag-planner` แล้ว โดยอ่านรูปแบบ `graph.json` และคำสั่ง `team.py` จาก **./base_skills/agent-team** ทดสอบใน container ได้ **PASS 8 / 8** (ข้อ 7 ใช้ตัวจำลอง ไม่ใช้โทเคน)

ข้อ	ทดสอบ	ผล	หลักฐาน
1	ติดตั้ง + `doctor`	PASS	yolo1–5 และ cyolo1–5 เห็น `SKILL.md` · doctor พบ base skills ครบ 3 ตัว · บัญชีพร้อม yolo1–3, cyolo1–2
2	`dag` จับข้อผิดพลาด	PASS	วงวน · id ซ้ำ · อ้าง id ที่ไม่มี · skill ไม่รู้จัก ครบ 4 แบบ
3	แผน 8 บล็อก ครบ 6 สกิล	PASS	สายวิกฤต B01 → B02 → B05 → B07 → B08 ยาว 7.2M tok · rank ตรงกับการคำนวณอิสระทุกบล็อก
4	`plan` จากโควตาจริง	PASS	ทุกบล็อกได้บัญชีฝั่งที่ถูก · review: yolo3 ทำ, cyolo2 ตรวจ · ไม่มีบัญชีใดเกิน slot
5	what-if `--accounts yolo1:5,…`	PASS	yolo1 ไม่ได้บล็อกเลย
6	`export`	PASS	`validate_graph()` ของ base skill รับ 8 node · `team.py run --dry-run` รับกราฟ
7	`run` → `report` ด้วยตัวจำลอง	PASS	`MA_SIMULATE_LIMIT`: 0 โทเคน · สถานะ failed/blocked ตรงกับ state.json · tokens 0 = ledger 0
8	เรียกจาก host	PASS	ผ่าน `docker exec` · และ `dag` + `plan` แบบสมมติบน host ด้วย `./base_skills`

- ไฟล์ที่สร้าง: `SKILL.md` · `scripts/dagplan.py` `kcquest.py` `render.py` · `references/blocks.md` `kcquest.md` `run.md`
- ไม่ได้แก้ไฟล์ใน `./base_skills` · **ยังไม่ได้รันทีมจริงด้วยโทเคน** (ทำเมื่อผู้ใช้สั่ง)

จัดหน้าใหม่จากผลทดสอบจริง · ข้อ 7 ใช้ตัวจำลอง MA_SIMULATE_LIMIT (0 โทเคน) และเจอข้อบกพร่อง 1 จุดใน report ซึ่งแก้แล้ว · รายงานเต็ม _solution/TEST-REPORT.md

ขั้นที่ 1 · ภาพจริงจากการทดสอบ: แผน 8 บล็อกครบ 6 สกิล



ทุกกล่องมีป้ายสีของสกิล + รุ่น/effort · ป้ายล่าง = บัญชีที่ KC-QUEST แจก · บล็อกหลายบัญชีมีหลายป้าย

ขั้นที่ 1 · ไฟล์ที่ Claude สร้าง

```
skills/dag-planner/
├── SKILL.md          ← base skills ที่ใช้ · ตาราง "ต้องการ → คำสั่ง" · ขั้นตอน · กติกาแตกบล็อก · ตารางเลือกสกิล · เรียกจาก host
├── scripts/          ← dagplan.py (คำสั่งหลัก) · kcquest.py (KC-QUEST ที่ดึงจาก orch.py) · render.py (วาด DAG · Gantt · compare)
└── references/       ← blocks.md (ฟิลด์ + ตัวอย่าง 8 บล็อก) · kcquest.md (สูตรย่อ) · run.md (รันจริงและรายงาน)
```

เฉลยอยู่ที่ _solution/skills/dag-planner/ · สคริปต์หา base skills ตามลำดับ: DAGPLAN_BASE_SKILLS → โฟลเดอร์ข้างเคียงใน shared/skills → ./base_skills

2

ติดตั้งด้วยการคัดลอกโฟลเดอร์

คัดลอก 4 โฟลเดอร์ **agent-team · yolo · cyolo · dag-planner** ลง shared/skills ทั้งสองฝั่ง · ไม่ต้องรันสคริปต์ติดตั้ง

ขั้นที่ 2 · พรอมต์

```
ติดตั้งสกิลลง container multi-account ด้วยการคัดลอกโฟลเดอร์อย่างเดียว
- คัดลอก ./base_skills/agent-team ./base_skills/yolo ./base_skills/cyolo และ ./skills/dag-planner ไปวางที่ /root/ai-accounts/.Claude/shared/skills และ /root/ai-accounts/.Codex/shared/skills ใน container (base skill ตัวใดที่มีอยู่แล้วและตรงกันทุกไฟล์ ไม่ต้องคัดลอกซ้ำ)
- เปิดแต่ละบัญชีหนึ่งครั้งด้วย yoloN --version และ cyoloN --version (wrapper จะเชื่อมสกิลให้แต่ละบัญชีเอง)
- ตรวจว่าทุกบัญชี yolo1-5 และ cyolo1-5 มองเห็น dag-planner/SKILL.md แล้วรันคำสั่ง doctor ของสกิล
สรุปเป็นตาราง สกิล | มาจาก | ตรงกับไฟล์บนเครื่องไหม และตาราง ฝั่ง | มองเห็นสกิล | ผล doctor แล้วหยุด
```

> base skills ต้องอยู่ข้าง dag-planner ใน shared/skills เดียวกันเสมอ · ใช้สกิลที่เตรียมไว้: เปลี่ยน ./skills/dag-planner เป็น ./_solution/skills/dag-planner

ขั้นที่ 2 · ผลจริง 10 พ.ย. 2026

✻ Claude · ขั้นที่ 2 · ติดตั้ง base skills + dag-planner · ผลจริง 10 พ.ย. 2026 (จัดหน้าใหม่ให้อ่านง่าย)

คัดลอก 4 โฟลเดอร์ลง `shared/skills` ของทั้งสองฝั่งด้วย `docker cp` แล้ว ไฟล์ใน container ตรงกับไฟล์บนเครื่องทุกไฟล์

สกิล	มาจาก	ฝั่ง Claude	ฝั่ง Codex
agent-team	`./base_skills`	ตรงกัน	ตรงกัน
yolo	`./base_skills`	ตรงกัน	ตรงกัน
cyolo	`./base_skills`	ตรงกัน	ตรงกัน
dag-planner	`./skills`	ตรงกัน	ตรงกัน

ฝั่ง	มองเห็น dag-planner/SKILL.md	ผล doctor
Claude	yolo1–5 ครบ 5/5	พร้อมใช้ ✅
Codex	cyolo1–5 ครบ 5/5	พร้อมใช้ ✅

- `doctor` : base skills อยู่ที่ `/root/ai-accounts/.Claude/shared/skills` · ตัววาด PNG พร้อม
- บัญชีพร้อม: yolo1, yolo2, yolo3, cyolo1, cyolo2 · บัญชีที่เหลือยังไม่ได้ล็อกอิน (ต้องล็อกอินเอง)

> 4 โฟลเดอร์ใน container ตรงกับไฟล์บนเครื่องทุกไฟล์ · yolo1–5 และ cyolo1–5 เห็น dag-planner/SKILL.md ครบ · doctor ผ่านทั้งสองฝั่ง

คัดลอกเองได้ด้วย docker cp ทั้ง 4 โฟลเดอร์ หรือลากวางใน JupyterLab ที่ .Claude/shared/skills และ .Codex/shared/skills

3

วางแผนโจทย์ "เหมียวช็อป" (ยังไม่รัน)

Claude Code app **เจอสกิล dag-planner** เอง → บล็อกงาน + DAG + Gantt แผน · จบที่แผน ยังไม่ให้ทีมลงมือทำ (82 วินาที)

โจทย์: เว็บร้านอาหารแมว เหมียวช็อป

ข้อมูลและโค้ดหน้าเว็บ `yolo` `cyolo`

รูปสินค้า `image`

สโลแกนหัวเว็บ `best_of`

หน้าตาเว็บที่ต้องมีคนตรวจ `review`

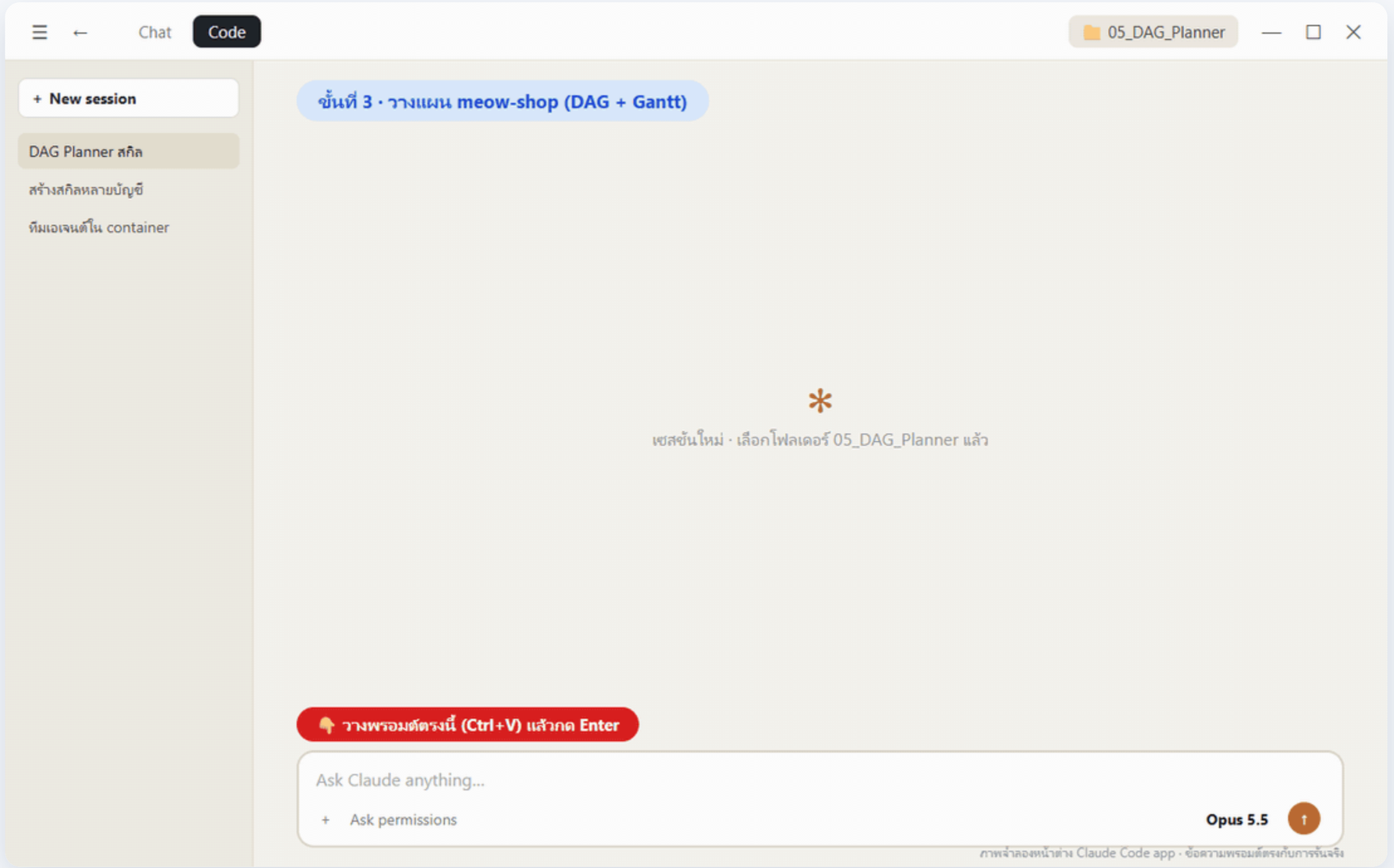
โจทย์เล็ก แต่ใช้สกิลหลายแบบ

ขั้นที่ 3 · พรอมต์

```
โจทย์: ทำหน้าเว็บร้านอาหารแมว "เหมียวช็อป" แบบหน้าเดียว (HTML + CSS + JavaScript ไม่ต้องมี server) มีสินค้า 6 รายการพร้อมรูป ตะกร้าสินค้าที่จำได้หลังรีเฟรช และสโลแกนหัวเว็บ แล้วทดสอบว่าใช้งานได้จริง
ชื่อแผน: meow-shop
จำนวน agent สูงสุด: 4

ใช้ทีมเอเจนต์ใน container multi-account วางแผนงานนี้ ทำงานใน container ผ่าน docker exec เท่านั้น ห้ามแก้ไฟล์ตั้งค่า ห้ามล็อกอินแทนฉัน
ก่อนเริ่ม ดูว่าใน container มีสกิลที่ช่วยงานนี้ได้หรือไม่ (/root/ai-accounts/.Claude/shared/skills) ถ้ามีให้อ่านและใช้
1. แตกโจทย์เป็นบล็อกงาน 6-10 บล็อก แต่ละบล็อกบอกสกิลที่ใช้ทำ ความยาก ไฟล์ผล และเกณฑ์เสร็จ
2. วาด DAG แล้ววางแผนว่าบล็อกไหนให้บัญชีไหน เมื่อไร (Gantt) จากโควตาจริง ยังไม่ต้องรันงานจริง
ตอบภาษาไทยสั้น ๆ: ตาราง บล็อก | ชื่อ | สกิล | ความยาก | ต้องรอ | rank · สายวิกฤตพร้อมเหตุผล 1 บรรทัด · ตาราง บล็อก | บัญชี | เริ่ม-จบ (นาที) · path ของ dag.png และ gantt-plan.png
```

ขั้นที่ 3 · 💻 Claude Code app



ขั้นที่ 3 · ผลจริง 8 พ.ย. 2026 · 8 บล็อก 5 สกิล · แผน 1.7 นาที

✻ **Claude** · ขั้นที่ 3 · วางแผน meow-shop (DAG + Gantt) · คำตอบจริง (จัดหน้าใหม่ให้อ่านง่าย)

วางแผน meow-shop เสร็จแล้ว ยังไม่ได้รันงานจริง ในโฟลเดอร์ shared มีสกิล **dag-planner** ตรงกับงานนี้ จึงอ่านและใช้ตามขั้นตอนของสกิล (`doctor` → `dag` → `plan`)

บล็อก	ชื่อ	สกิล	ความยาก	ต้องรอ	rank
B01	โครงโปรเจกต์ + index.html	cyolo	1	–	3.6
B02	ข้อมูลสินค้า 6 รายการ (products.js)	yolo	2	–	3.6
B03	ภาพสินค้า 6 ช่องในภาพเดียว 3×2 (products.png)	image	1	–	3.6
B04	สโลแกนหัวเว็บ (slogan.txt)	best_of n=2	1	–	2.1
B05	สไตล์ CSS (style.css)	review	2	B01	3
B06	ตะกร้า + localStorage (cart.js)	yolo	3	B02	3
B07	แสดงการ์ดสินค้า (shop.js)	cyolo	2	B01, B02, B03	3
B08	รวม + ทดสอบจริงด้วย Playwright (TEST.md)	yolo	3	B04, B05, B06, B07	1.5

สายวิกฤต: B01 → B05 → B08 (ยาว 3.6M โทเคน) เพราะ B05 เป็นงาน review ขนาด M ที่ต้องมีคนทำและคนตรวจ และ B08 ต้องรอทุกบล็อก

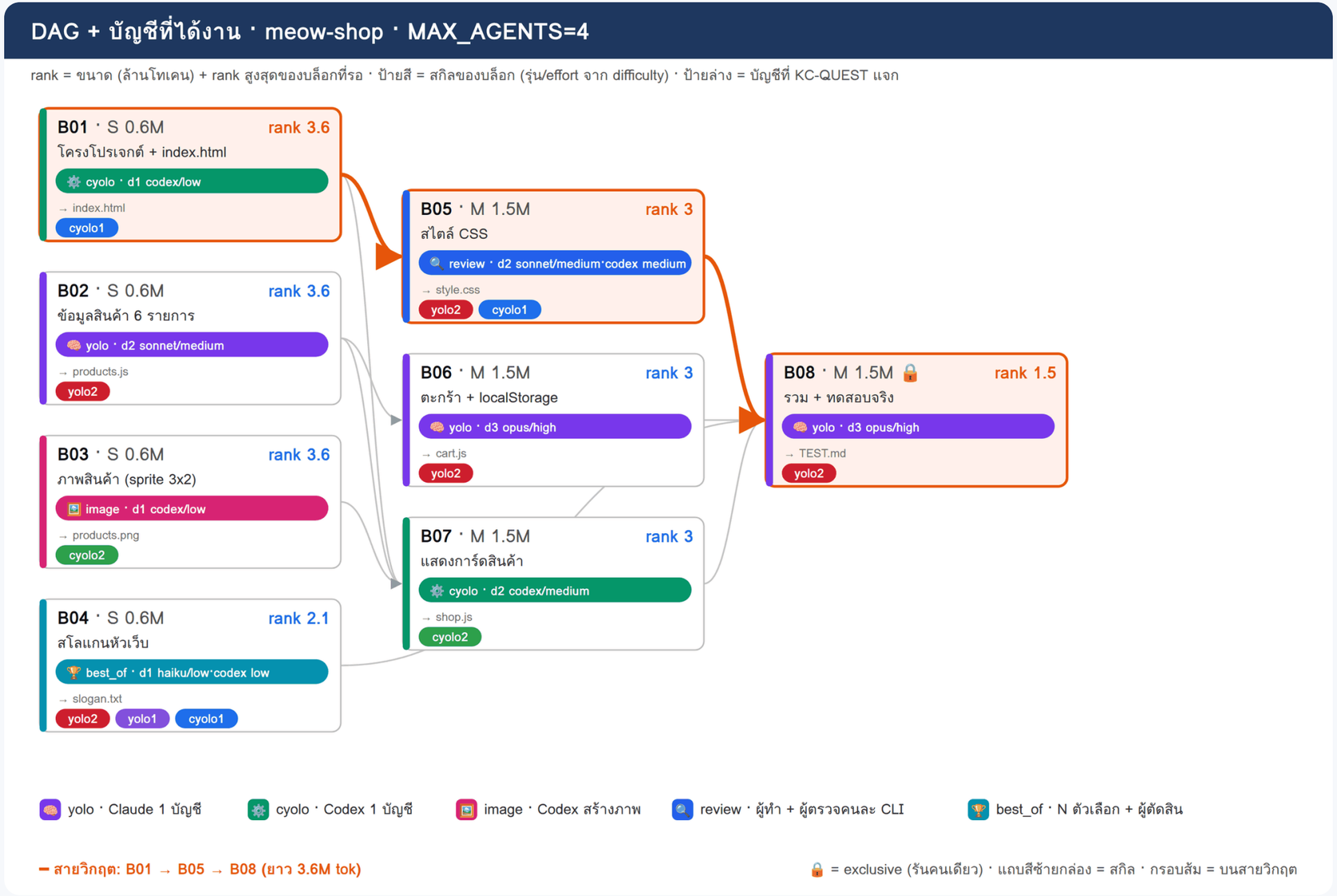
บล็อก	บัญชี	เริ่ม–จบ (นาที)
B01	cyolo1	0.0–0.2
B02	yolo2	0.0–0.1
B03	cyolo2	0.0–0.5
B06	yolo2	0.1–0.5
B05	yolo2 (ทำ) + cyolo1 (ตรวจ)	0.2–1.0
B07	cyolo2	0.5–1.1
B04	yolo2 + yolo1 (ตอบ) + cyolo1 (ตัดสิน)	1.0–1.3
B08	yolo2	1.3–1.7

- agent พร้อมกันสูงสุด 4 ตัว ใช้เวลารวมประมาณ **1.7 นาที**
- ระบบเลือก yolo2 บ่อยที่สุด เพราะโควตาจะรีเซ็ตเร็วที่สุด (26.6 ชม.) ใช้ให้คุ้มก่อนรีเซ็ต

ไฟล์ผล: `./workspace/plans/meow-shop/dag.png` · `gantt-plan.png` · `PLAN.md` (เหตุผลที่เลือกแต่ละบัญชี)

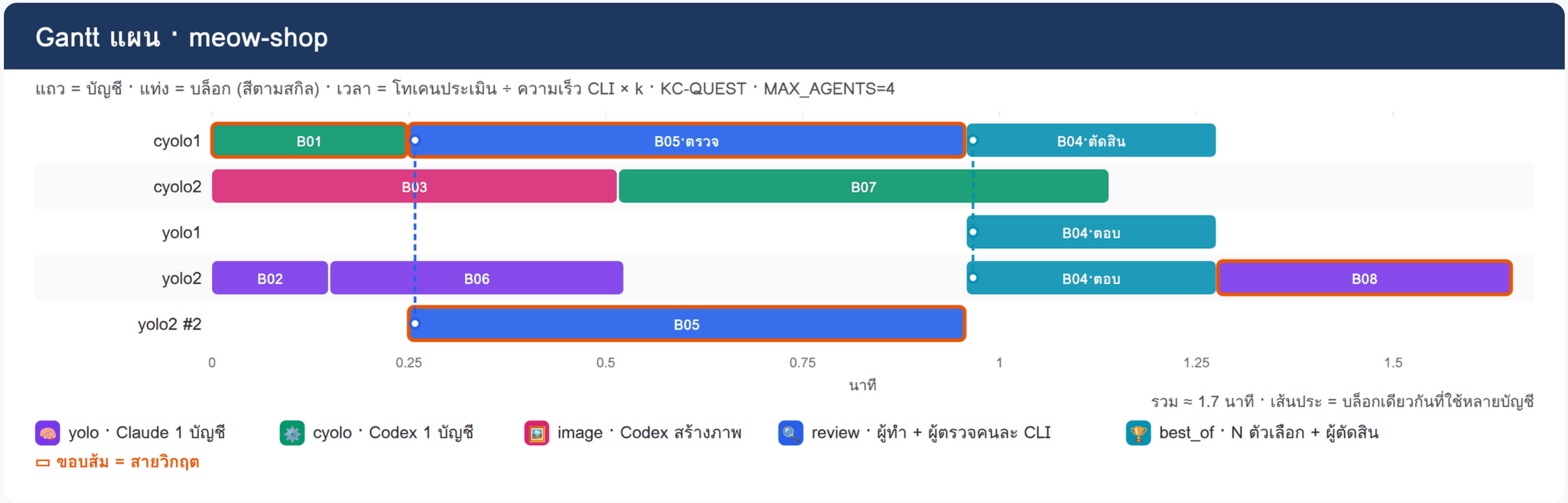
ถ้าจะรันจริงต่อ ใช้คำสั่ง `export` , `run` , `report` ตามลำดับ ซึ่งจะใช้โทเคนจริง ให้บอกก่อนแล้วจะเริ่มรัน

ขั้นที่ 3 · dag.png (ภาพจริง)



สายวิกฤต B01 → B05 → B08 = 0.6 + 1.5 + 1.5 = 3.6 ล้านโทเคน · B05 review: yolo2 ทำ + cyolo1 ตรวจ · B04 best_of: 3 บัญชี

ขั้นที่ 3 · gantt-plan.png (ภาพจริง)



บล็อกหลายบัญชีมีแท่งหลายแถวเชื่อมด้วยเส้นประ · yolo2 ได้งานมากสุดเพราะโควตาจะรีเซ็ตเร็วสุด (26.6 ชม.)

ขั้นที่ 3 · อ่านตัวเลขในแผนอย่างไร

ตัวเลขนาทีขึ้นกับ k · บัญชีขึ้นกับโควตา

มี calib.json → 1.7 นาที

ผลในภาพใช้ k จากการรันทดสอบเล็กก่อนหน้า

ไม่มี calib.json (k = 1) → ราว 45 นาที

เครื่องที่ยังไม่เคยรันจริง แผนเดียวกันได้สเกลเวลาต่างกัน

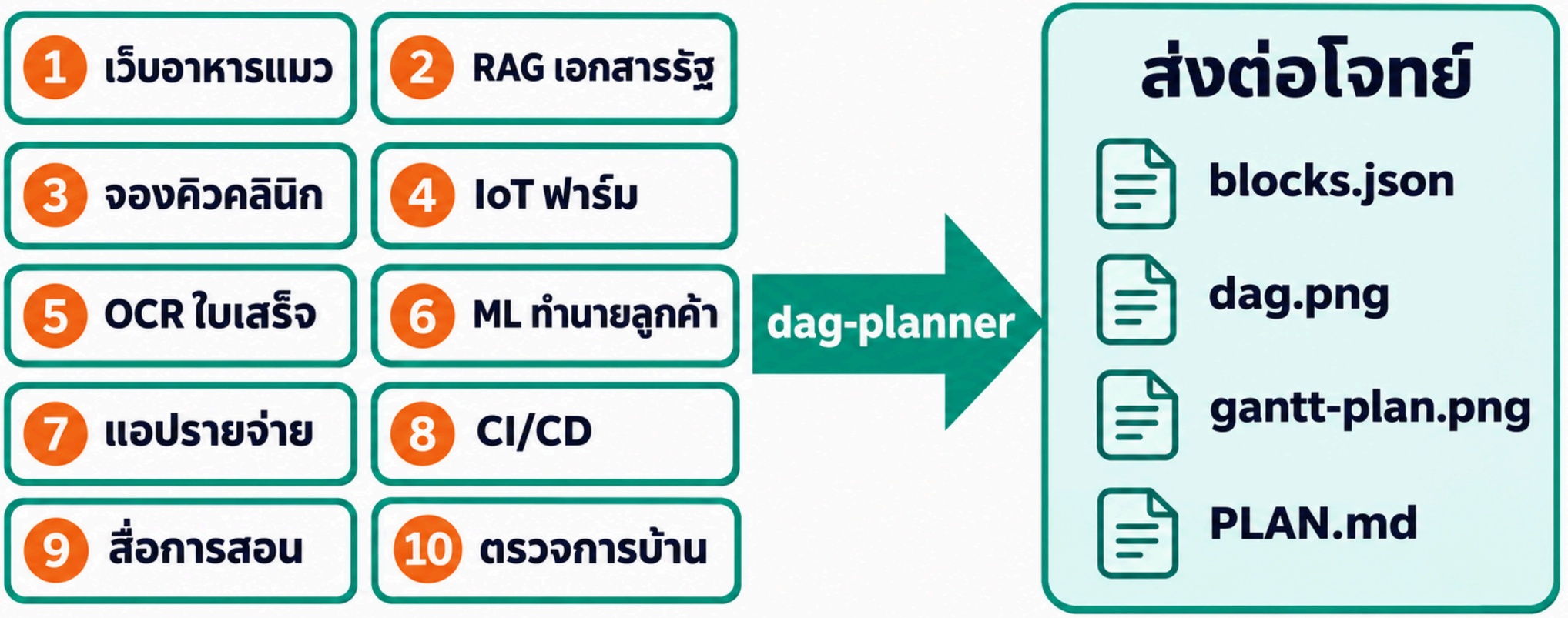
ลำดับงาน สายวิกฤต และวิธีแจกบัญชีเหมือนเดิม ต่างกันแค่สเกลเวลา · บัญชีที่ได้งานขึ้นกับโควตา ณ เวลาที่สั่ง plan ผลบนเครื่องของแต่ละคนจึงไม่จำเป็นต้องตรงกับภาพ

ส่วนที่ 5 จาก 6

การบ้าน: วางแผน 10 โจทย์

ทำแบบขั้นที่ 3 กับโจทย์ 10 ข้อ ได้ **DAG + Gantt** ข้อละชุด · ยังไม่รันจริง

การบ้าน: 10 โจทย์ วางแผนด้วย dag-planner



วางแผนเท่านั้น ห้ามรันจริง

การบ้าน · โจทย์ 10 ข้อ (ชื่อแผนตามตาราง)

ข้อ	ชื่อแผน	โจทย์ (ย่อ)
1	`hw01-catfood-next`	เว็บขายอาหารแมว ใช้ Next.js กับ Docker มีแชทบอทช่วยแนะนำอาหาร
2	`hw02-gov-rag-k8s`	RAG ถามตอบเอกสารภาครัฐ deployment ด้วย Kubernetes
3	`hw03-clinic-line-queue`	จองคิวคลินิกผ่าน LINE chatbot มีแจ้งเตือนและ dashboard
4	`hw04-iot-farm-dashboard`	IoT ฟาร์มผัก MQTT → ฐานข้อมูล time series → Grafana + แจ้งเตือน
5	`hw05-receipt-ocr-api`	API อ่านใบเสร็จภาษาไทย (OCR) ด้วย FastAPI + คิวงาน + Docker
6	`hw06-churn-ml-pipeline`	pipeline ทำนายลูกค้าเลิกใช้บริการ ถึง API และ dashboard
7	`hw07-expense-mobile`	แอปมือถือบันทึกรายจ่าย สแกนสลิป มี backend และกราฟสรุป
8	`hw08-cicd-microservices`	แยก monolith เป็น 3 microservices + CI/CD + Helm บน Kubernetes
9	`hw09-course-media`	สื่อการสอนออนไลน์ 1 บท: สไลด์ ภาพ สคริปต์ แบบทดสอบ หน้าเว็บ
10	`hw10-ai-autograder`	ระบบส่งการบ้านเขียนโปรแกรม ตรวจใน sandbox และให้ AI เขียนคำแนะนำ

การบ้าน · พรอมต์ต่อ 1 โจทย์ (เปลี่ยน 2 บรรทัดแรกตามตาราง)

```
โจทย์: <คัดลอกโจทย์จากตาราง>
ชื่อแผน: <ชื่อแผนจากตาราง>
จำนวน agent สูงสุด: 4

ใช้ทีมเอเจนต์ใน container multi-account วางแผนงานนี้ ทำงานใน container ผ่าน docker exec เท่านั้น ห้ามแก้ไฟล์ตั้งค่า ห้ามล็อกอินแทนฉัน
ก่อนเริ่ม ดูว่าใน container มีสกิลที่ช่วยงานนี้ได้หรือไม่ (/root/ai-accounts/.Claude/shared/skills) ถ้ามีให้อ่านและใช้
1. แตกโจทย์เป็นบล็อกงาน 8-12 บล็อก แต่ละบล็อกบอกสกิลที่ใช้ทำ ความยาก ไฟล์ผล และเกณฑ์เสร็จ
2. วาด DAG แล้ววางแผนว่าบล็อกไหนให้บัญชีไหน เมื่อไร (Gantt) จากโควตาจริง
วางแผนเท่านั้น ห้ามสั่ง export run หรือ report
เสร็จแล้วคัดลอก blocks.json dag.png gantt-plan.png และ PLAN.md ของแผนนี้มาไว้ที่ ./homework/<ชื่อแผน>/ บนเครื่อง
ตอบภาษาไทยสั้น ๆ: ตาราง บล็อก | ชื่อ | สกิล | ความยาก | ต้องรอ | rank · สายวิกฤตพร้อมเหตุผล 1 บรรทัด · ตาราง บล็อก | บัญชี | เริ่ม-จบ (นาที)
```

บัญชีไม่พร้อมหรือโควตาหมด: เพิ่มบรรทัด `ใช้โควตาสมมติ: --accounts yolo1:80,yolo2:60,cyolo1:90,cyolo2:70` สกิลจะไม่เรียกบัญชีใดเลย

การบ้าน · สิ่งที่ต้องส่งและเกณฑ์ให้คะแนน

ต่อโจทย์ 5 ไฟล์

ไฟล์	คืออะไร
`blocks.json`	บล็อกงานที่ AI แตก (แก้เองได้)
`dag.png`	DAG
`gantt-plan.png`	Gantt แผน
`PLAN.md`	ตารางแผนและเหตุผลที่เลือกบัญชี
`ANSWER.md`	คำตอบ 4 ข้อ เขียนเอง

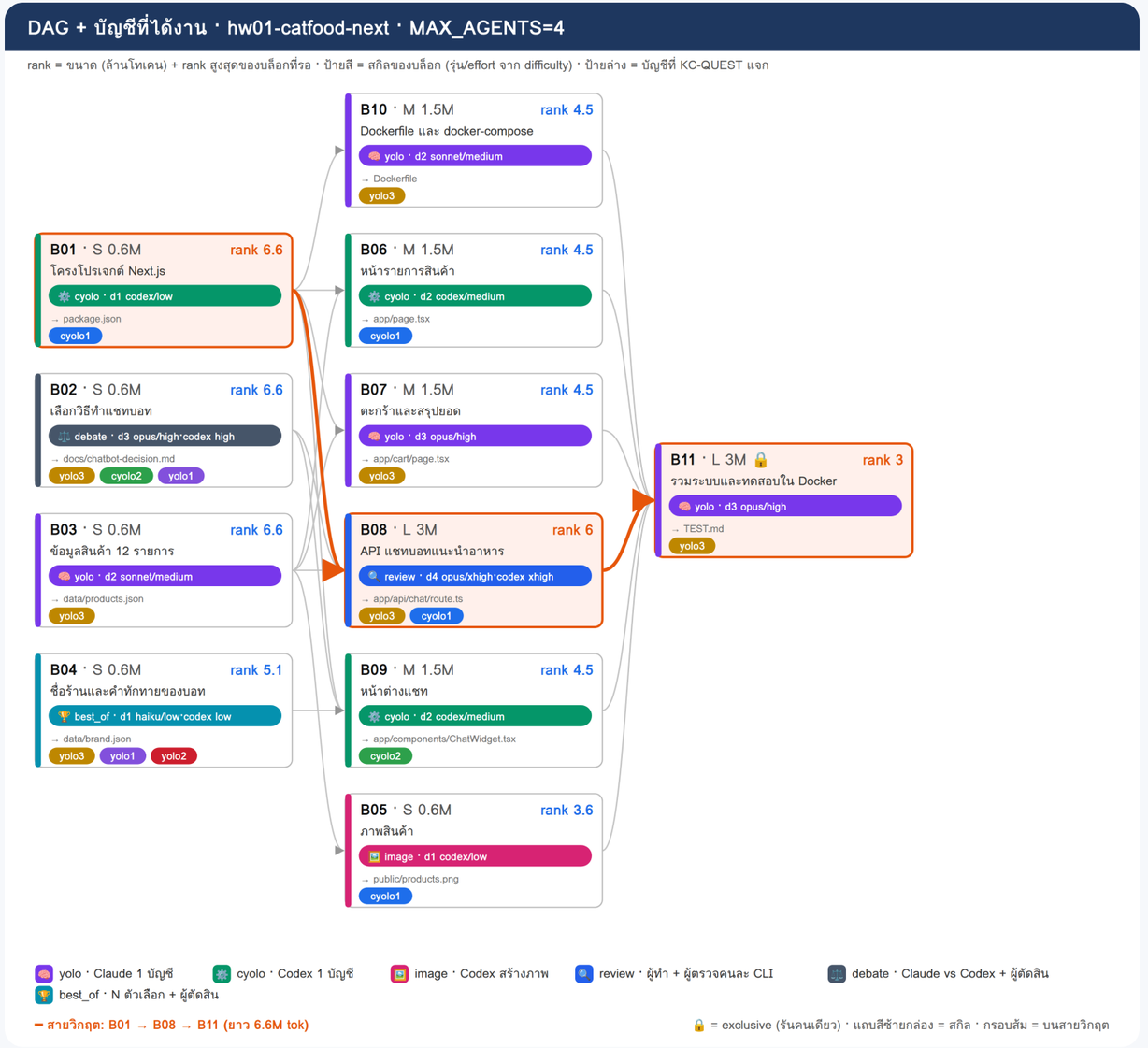
ANSWER.md: 1 สายวิกฤตและคิด rank ด้วยมือ · 2 เหตุผลที่ใช้ review · debate · best_of · image · 3 ลูกศรที่ตัดไม่ได้ และคู่ที่ตั้งใจไม่ใส่ลูกศร · 4 ลด agent สูงสุดเหลือ 2 แล้วเกิดอะไร

ข้อละ 10 คะแนน รวม 100

เกณฑ์	คะแนน
ไฟล์ครบ และ dag ไม่มี ERROR	2
บล็อกสมเหตุผล 8–12 บล็อก ไม่ใหญ่เกิน L	2
ลูกศรไม่ขาดไม่เกิน บล็อกสุดท้ายเป็น exclusive	2
เลือกสกิลเหมาะกับงาน	2
ANSWER.md ครบ คิด rank ถูก	2

ห้ามสั่ง `export` · `run` · `report` การบ้านจบที่แผน

การบ้าน · ตัวอย่างข้อ 1: เว็บอาหารแมว Next.js + Docker + แชทบอท (11 บล็อก ครบ 6 สกิล)



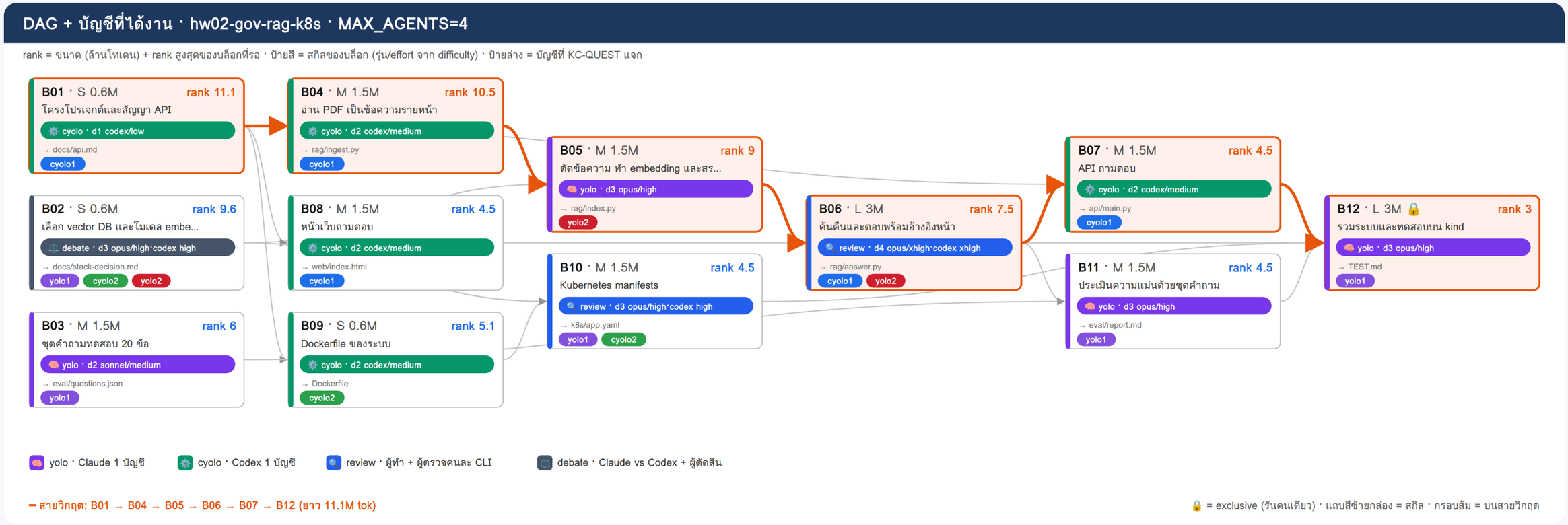
ภาพสินค้า B05 รอข้อมูลสินค้า B03 · หน้าต่างแชท B09 ไม่รอ API แชท B08 เพราะยึดรูปแบบจากไฟล์ตัดสินใจของ B02 · สายวิกฤต B01 → B08 → B11 = 0.6 + 3 + 3 = 6.6

เฉลยการบ้าน · ผลจริงจากสกิล 10 พ.ย. 2026 (โควตาสมมติ · ไม่ได้รันทีม)

ข้อ	แผน	บล็อก	สายวิกฤต	ยาว (M tok)	agent 4	agent 2
1	hw01-catfood-next	11	B01 → B08 → B11	6.6	110.5 นาที	ได้ 6/11
2	hw02-gov-rag-k8s	12	B01 → B04 → B05 → B06 → B07 → B12	11.1	127.5 นาที	ได้ 5/12
3	hw03-clinic-line-queue	10	B02 → B05 → B10	7.5	83.4 นาที	ได้ 7/10
4	hw04-iot-farm-dashboard	10	B02 → B05 → B07 → B10	5.7	70.9 นาที	ได้ 3/10
5	hw05-receipt-ocr-api	11	B02 → B05 → B06 → B08 → B11	9.6	101.5 นาที	ได้ 5/11
6	hw06-churn-ml-pipeline	10	B02 → B04 → B05 → B06 → B10	9.6	100.0 นาที	ได้ 4/10
7	hw07-expense-mobile	10	B01 → B04 → B06 → B10	8.1	101.5 นาที	ได้ 1/10
8	hw08-cicd-microservices	11	B01 → B02 → B03 → B11	7.5	96.9 นาที	ได้ 0/11
9	hw09-course-media	10	B01 → B03 → B08 → B10	6.6	89.8 นาที	ได้ 7/10
10	hw10-ai-autograder	11	B02 → B05 → B11	6.6	89.0 นาที	ได้ 7/11

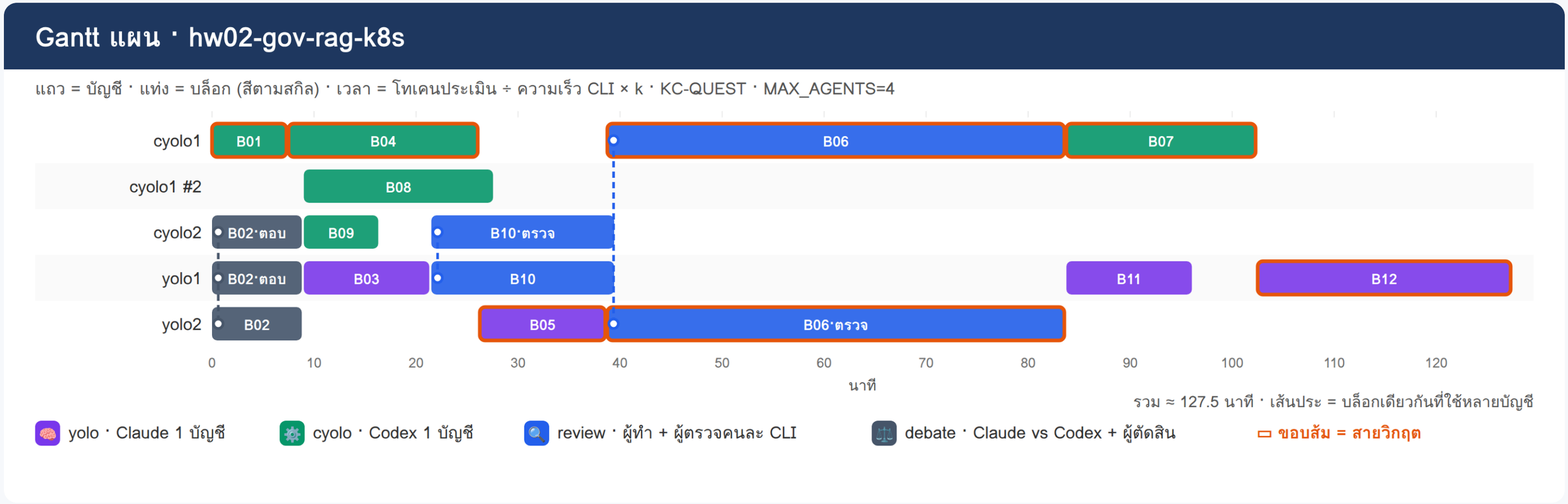
เฉลยเต็ม (พรอมต์ → บล็อก → DAG → Gantt → ANSWER.md) อยู่ที่ _solution/homework/ · เวลาเป็นค่าประเมินก่อนรันจริง (k = 1) · แผนของผู้เรียนไม่จำเป็นต้องเหมือนเฉลย

เฉลยข้อ 2 · RAG ถามตอบเอกสารภาครัฐบน Kubernetes · dag.png



12 บล็อก สายวิกฤตยาว 6 บล็อก เพราะ ingest → ทำดัชนี → ตอบ → API ต้องทำต่อกัน · ชุดคำถามทดสอบ B03 หน้าเว็บ B08 และ Dockerfile B09 ทำขนานได้

เฉลยข้อ 2 · gantt-plan.png (โควตาสมมติ · agent สูงสุด 4)



บล็อก debate B02 และ review B06 B10 มีแท่งหลายแถวเชื่อมด้วยเส้นประ · ขอบส้มคือสายวิกฤต · รวมประมาณ 127.5 นาที (ค่าประเมิน k = 1)

เฉลยการบ้าน · คำถามข้อ 4

ลด agent สูงสุดเหลือ 2 แล้ววางแผนไม่ครบทุกข้อ

เหตุ

ทุกโจทย์มีบล็อก `debate` หรือ `best_of` ที่ต้องใช้ 3 บัญชีพร้อมกัน เกินเพดาน 2 จึงไม่มีบัญชีรับ และบล็อกที่รออยู่ถูกบล็อกตาม

ตัวอย่าง

ข้อ 8 ได้ 0/11 เพราะบล็อกแรกเป็น `debate` · ข้อ 3 ได้ 7/10 เพราะ `best_of` อยู่ปลายทาง

ตัวเลขเวลารวมที่ "สั้นลง" ในรอบ agent 2 ใช้เทียบไม่ได้ เพราะแผนไม่ครบ · ทางแก้: เปลี่ยนสกิลของบล็อกนั้นเป็น `yolo` หรือ `review` หรือใช้ agent อย่างน้อย 3

ส่วนที่ 6 จาก 6

ภาคต่อ: รันแผนจริงและ what-if

ยังไม่ทำในคาบนี้ เพราะใช้โทเคนของทีม · ผลทั้งหมดมาจากการรันจริง 8 พ.ย. 2026 ด้วย agent-team รุ่นก่อน

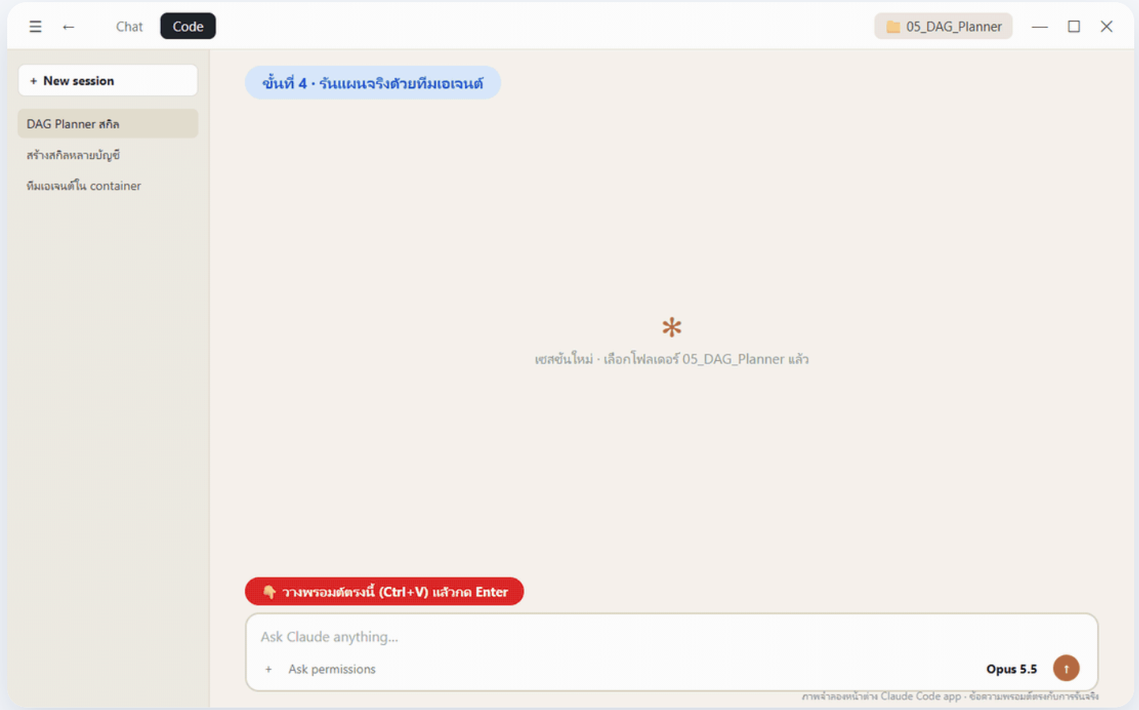
4

รันแผนจริงด้วยทีมเอเจนต์

export → run → report · ทีมรันจริง **2.8 นาที** · เสร็จ 8/8 บล็อก

ภาคต่อ · ขั้นที่ 4 · 💻 Claude Code app

```
ชื่อแผน: meow-shop
รันแผนนี้จริงด้วยทีมเอเจนต์ใน container multi-account ทำงานผ่าน
docker exec เท่านั้น ห้ามแก้ไฟล์ตั้งค่า ห้ามล็อกอินแทนฉัน
- ใช้แผนที่วางไว้แล้ว (ไม่ต้องวางแผนใหม่) ส่งให้ agent-team รันจนจบด้วย
สกิล dag-planner แล้วทำรายงานเทียบแผนกับของจริง
- ถ้าบล็อกใดล้ม ให้รายงานตามจริง ห้ามแก้แทนหรือเดาผล
ตอบภาษาไทยสั้น ๆ: ตาราง บล็อก | สกิล | บัญชีแผน → จริง | นาทีแผน →
จริง | tokens | สถานะ · เวลารวมแผนเทียบของจริง · path ของ gantt-
actual.png, compare.png และหน้าเว็บที่ได้
```



"ใช้แผนที่วางไว้แล้ว" สำคัญ ไม่เช่นนั้น Claude อาจวางแผนใหม่ทับแผนขั้นที่ 3

ภาคต่อ · ขั้นที่ 4 · ผลจริง

✻ **Claude** · ขั้นที่ 4 · รันแผนจริงด้วยทีมเอเจนต์ · คำตอบจริง (จัดหน้าใหม่ให้อ่านง่าย)

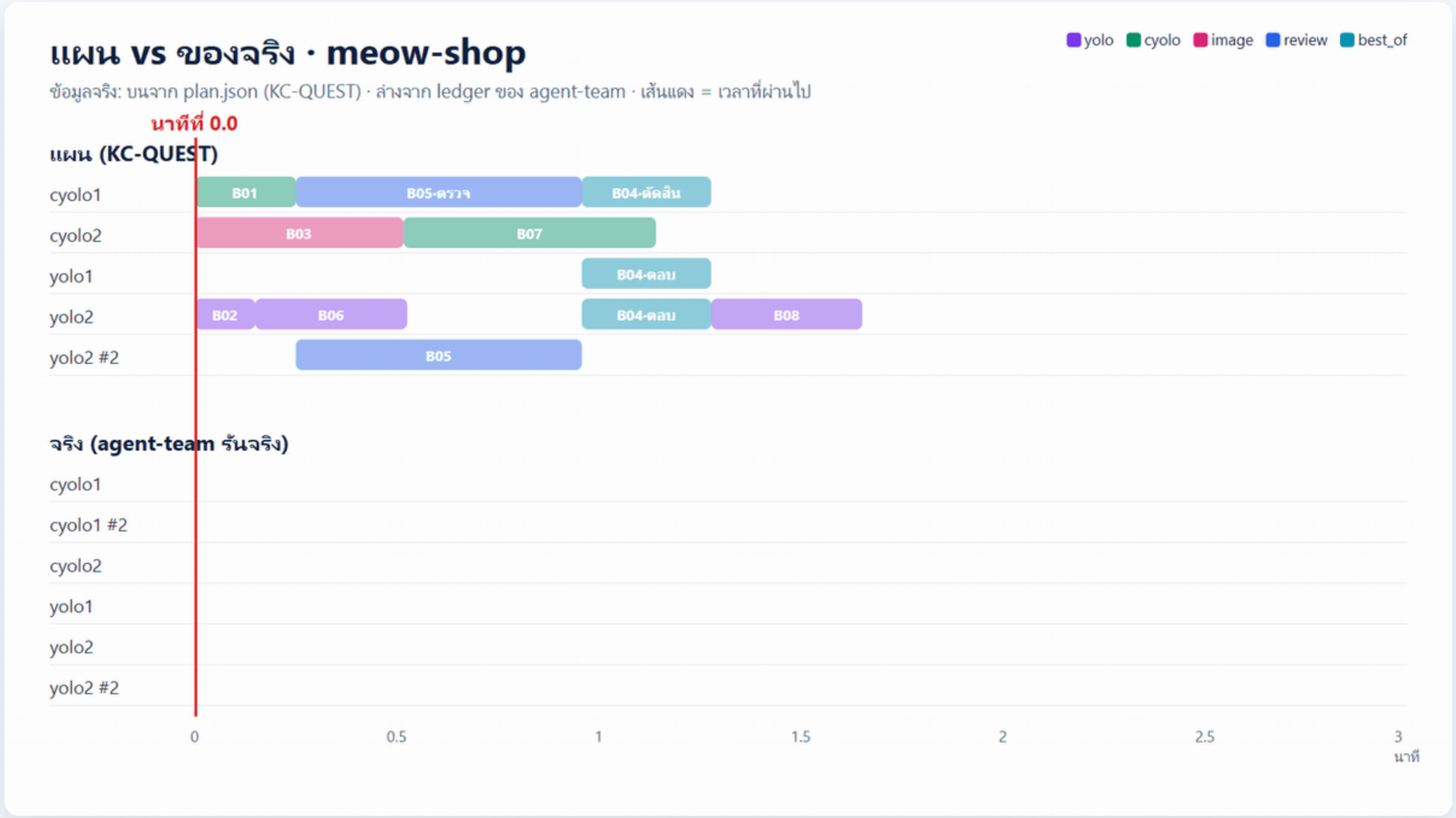
รันแผน meow-shop จบแล้ว ✅ เสร็จ 8/8 บล็อก ไม่มีบล็อกล้ม

บล็อก	สกิล	บัญชีแผน → จริง	นาทีแผน → จริง	tokens	สถานะ
B01	cyolo	cyolo1 → cyolo1	0.25 → 0.45	59,754	done
B02	yolo	yolo2 → yolo2	0.15 → 0.27	86,606	done
B03	image	cyolo2 → cyolo2	0.52 → 0.63	170,862	done
B04	best_of	yolo2+yolo1, ตัดสิน cyolo1 → ตรงแผน	0.32 → 0.27	98,997	done
B05	review	yolo2 ทำ, cyolo1 ตรวจ → ตรงแผน	0.71 → 0.57	166,422	done
B06	yolo	yolo2 → **yolo1**	0.37 → 0.88	171,320	done
B07	cyolo	cyolo2 → **cyolo1**	0.62 → 0.70	106,023	done
B08	yolo	yolo2 → **yolo1**	0.38 → 1.50	464,823	done

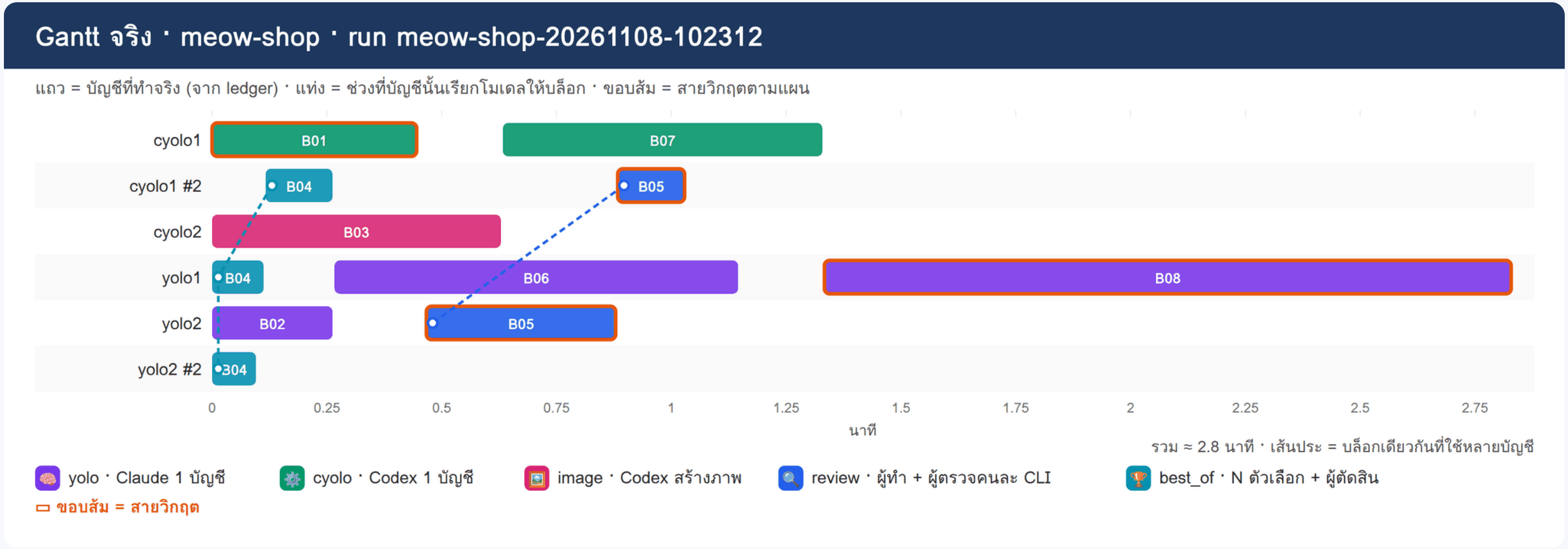
- **เวลารวม:** แผน 1.7 นาที → จริง 2.8 นาที ช้ากว่าแผนส่วนใหญ่เพราะ B08 (รวม + ทดสอบ) ใช้ 1.5 นาที
- **tokens รวม:** 1,324,807 เท่ากับยอดใน ledger
- **บัญชีที่ไม่ตรงแผน:** B06, B07, B08 เพราะ export ส่งงานเป็นฝั่ง yolo/cyolo ไม่ผูกบัญชี agent-team จึงเลือกบัญชีเองตามโหลด
- **ผลทดสอบของ B08:** agent เขียนไว้ใน TEST.md ว่า PASS 6/6 ตะกร้ายังอยู่หลังรีเฟรช และไม่มี console error (ผลที่ agent รายงานเอง)
- **ตัวปรับเวลา k** อัปเดตใน `calib.json` เป็น `{"cyolo":0.0438,"yolo":0.0884,"image":0.0844,"best_of":0.0368,"review":0.0313}` ซึ่ง plan ครั้งถัดไปจะใช้

ไฟล์: `./workspace/plans/meow-shop/gantt-actual.png` · `compare.png` · `REPORT.md` · หน้าเว็บ `app/index.html`

ภาคต่อ · ขั้นที่ 4 · แผน vs ของจริงตามเวลา (จาก plan.json + ledger)

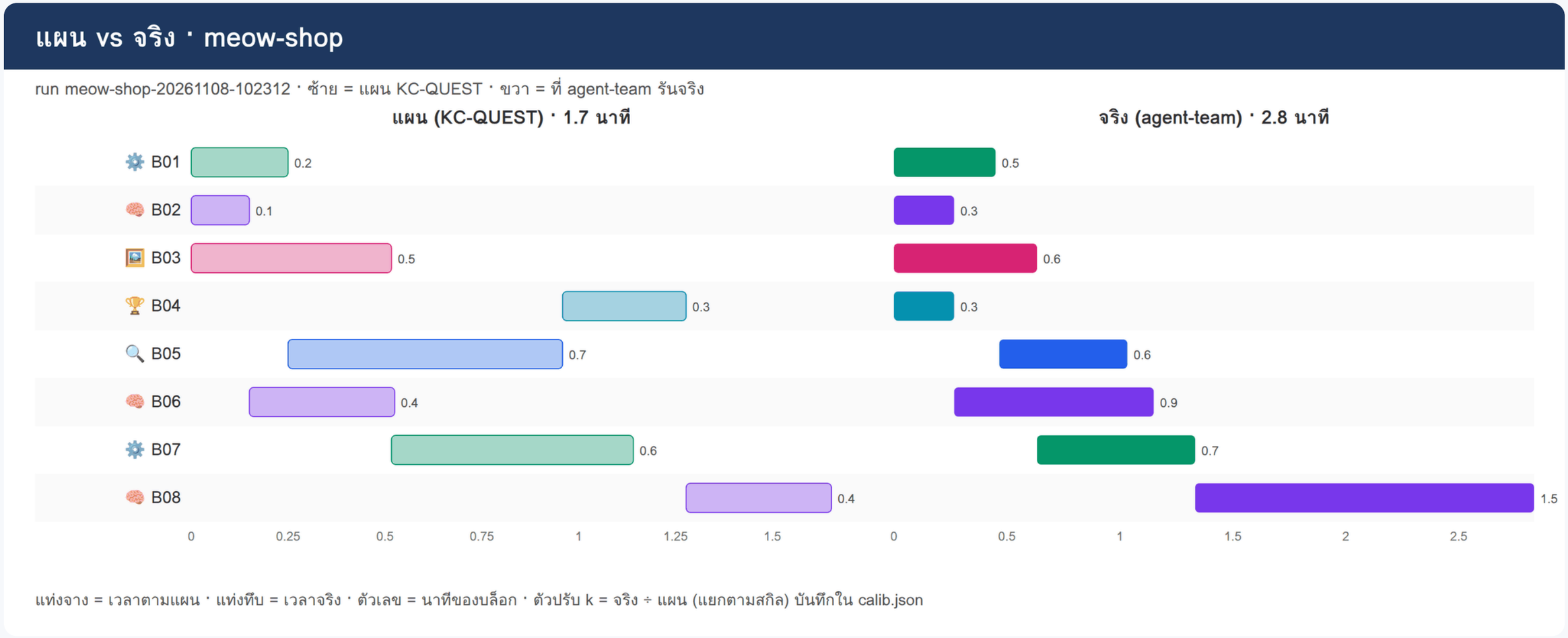


ภาคต่อ · ขั้นที่ 4 · gantt-actual.png (ภาพจริง)



review ของจริงเป็นลำดับ (ทำ แล้วตรวจ) · B06 B07 B08 ไปบัญชีอื่นเพราะ export ส่งเป็นฝั่ง · B08 ลากยาวถึงนาทีที่ 2.8

ภาคต่อ · ขั้นที่ 4 · compare.png (ภาพจริง)



B08 แผน 0.4 → จริง 1.5 นาที · k ใหม่: yolo 0.0884 · cyolo 0.0438 · image 0.0844 · best_of 0.0368 · review 0.0313

file:///workspace/plans/meow-shop/app/index.html

🐾 เหมียวช็อป

อาหารอร่อย สำหรับเจ้าเหมียวคนโปรด

อาหารแมวอร่อย เจ้าเหมียวชอบทุกวัน

เลือกซื้ออาหารแมว | ตะกร้าสินค้า

อาหารเม็ดแมว รสปลาทูน่า

อาหารเม็ดสำหรับแมวโต รสทูน่า โปรตีนสูง ขนาด 1.5 กก.

289 บาท

ใส่ตะกร้า

อาหารเปียก รสแซลมอน

อาหารเปียกซอง เนื้อแซลมอนชิ้นนุ่มในน้ำเกรวี่ 70 กรัม

25 บาท

ใส่ตะกร้า

ขนมแมวเลีย รสไก่

ขนมแมวเลียสุดฮิต รสไก่ เสริมทอรีน แพ็ก 4 ซอง

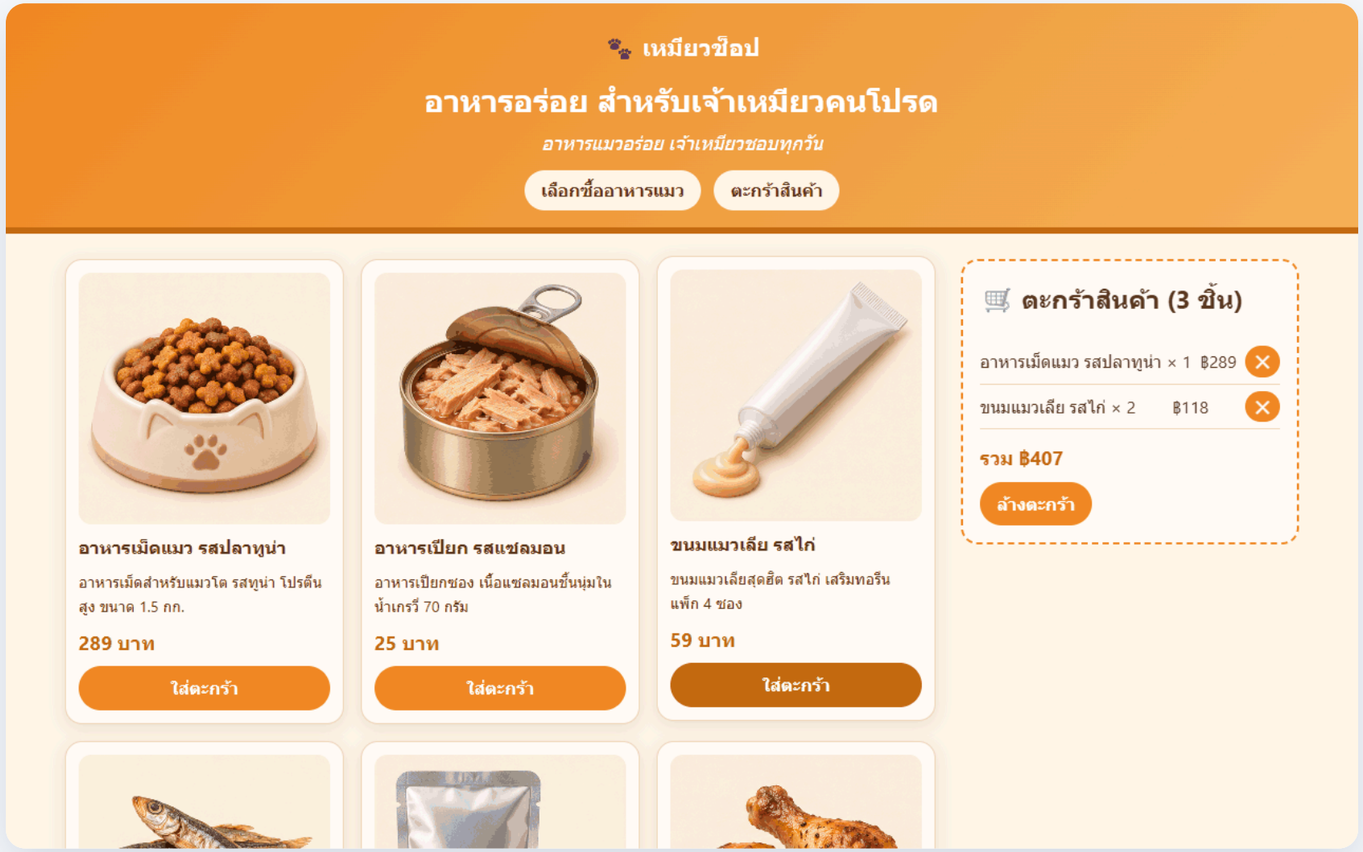
59 บาท

ใส่ตะกร้า

🛒 ตะกร้าสินค้า (0 ชิ้น)

ยังไม่มีสินค้าในตะกร้า

ขั้นที่ 4 · บทเรียนจากของจริง



ลูกศรที่ขาด = งานที่ไม่ตรงกัน

B08 บันทึกว่า **รูปบางช่องไม่ตรงชื่อสินค้า** เช่น "อาหารเปียก รสแซลมอน" ได้รูปกระป๋องทูน่า

สาเหตุ: B03 (ภาพ) **ไม่ได้รอ** B02 (ข้อมูลสินค้า) คนวาดภาพจึงไม่รู้ว่าแต่ละช่องคืออะไร · ตัดลูกศรได้เฉพาะงานที่ไม่ใช้ข้อมูลของกันจริง

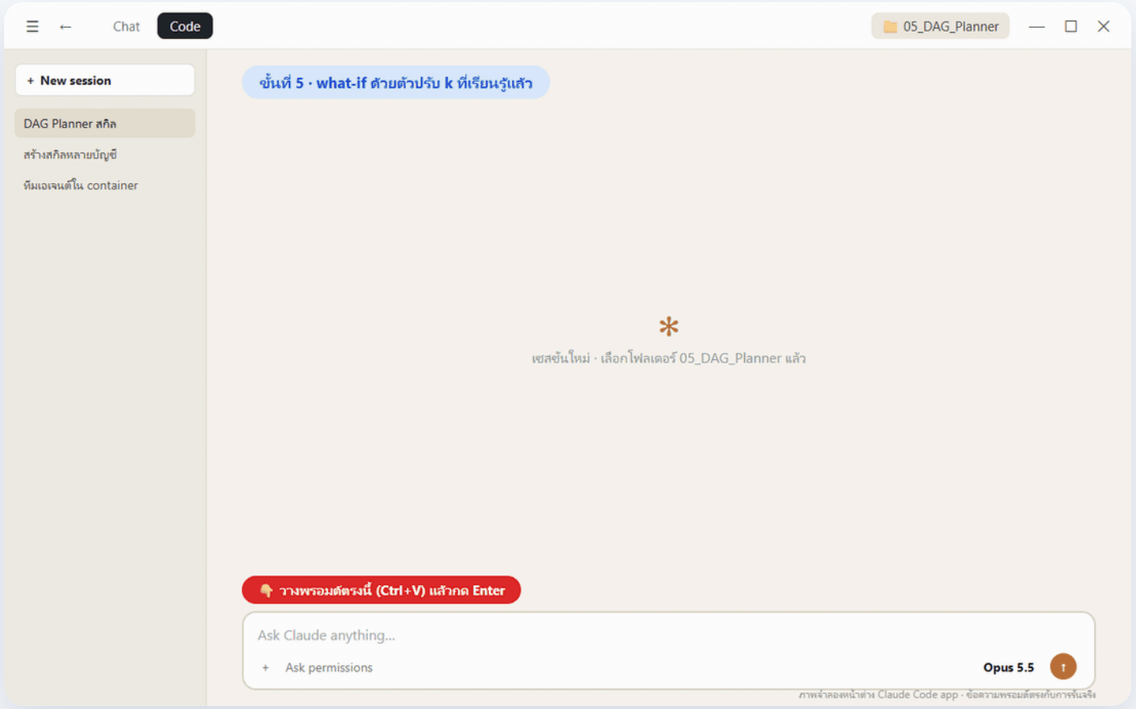
5

what-if ด้วย k ที่เรียนรู้แล้ว

plan ใหม่ 3 แบบ **ไม่รันทีมจริง** ไม่ใช้โทเคนของทีม (83 วินาที)

ภาคต่อ · ขั้นที่ 5 · 💻 Claude Code app

```
ชื่อแผน: meow-shop
วางแผนใหม่ 2 แบบ (ยังไม่รันจริง) โดยใช้ตัวปรับ k ที่ได้จากการรันจริงรอบที่แล้ว
แล้วเทียบกับแผนเดิม
ก. จำนวน agent สูงสุด 2
ข. yolo1 เหลือโควตาแค่ 5%
ทำผ่าน docker exec ใน container multi-account ด้วยสกิล dag-
planner
ตอบเป็นตาราง แบบ | เวลารวม (นาที) | บล็อกบนสายวิกฤตไปบัญชีไหน |
อะไรเปลี่ยน พร้อม path ของ gantt ทั้ง 2 แบบ
```

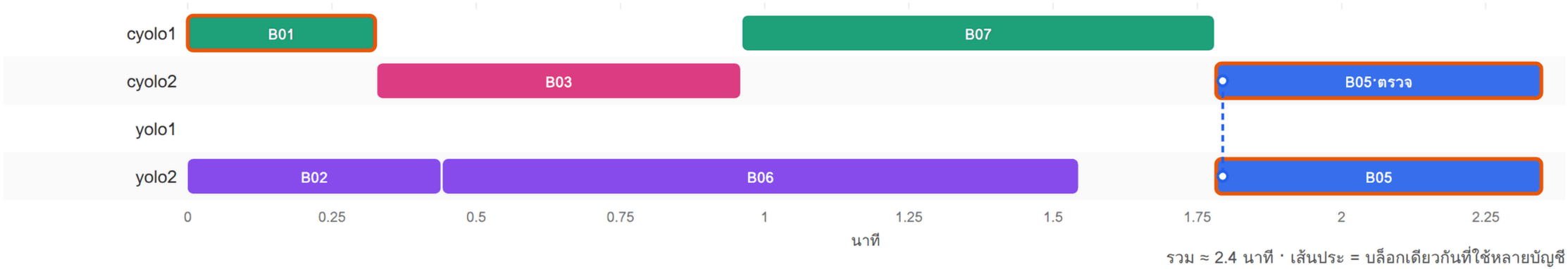


ภาคต่อ · ขั้นที่ 5 · ผลจริง

แบบ	เวลารวม	สิ่งที่เห็น
แผนเดิม (k จากขั้นที่ 1)	1.7 นาที	ประเมินต่ำไป
k ใหม่ · agent 4	**2.8 นาที**	**เท่ากับเวลาจริง**
ก. agent สูงสุด 2	2.4 นาที · เสร็จแค่ 6/8	B04 best_of ต้องใช้ 3 บัญชี → ไม่มีใครรับ · B08 ติดตาม
ข. yolo1 เหลือ 5%	2.9 นาที	yolo1 ไม่ได้บล็อกเลย สายวิกฤตไม่กระทบ

Gantt แผน · meow-shop

แถว = บัญชี · แท่ง = บล็อก (สีตามสกิล) · เวลา = โทเคนประเมิน ÷ ความเร็ว CLI × k · KC-QUEST · MAX_AGENTS=2



รวม ≈ 2.4 นาที · เส้นประ = บล็อกเดียวกันที่ใช้หลายบัญชี

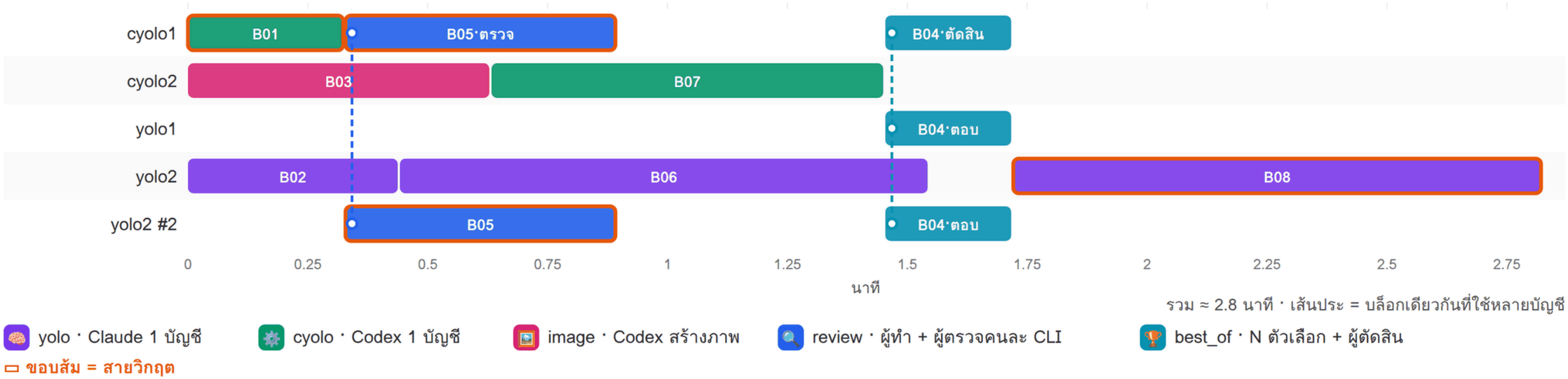
yolo · Claude 1 บัญชี · cyolo · Codex 1 บัญชี · image · Codex สร้างภาพ · review · ผู้ทำ + ผู้ตรวจคนละ CLI · best_of · N ตัวเลือก + ผู้ตัดสิน

▭ ขอบส้ม = สายวิกฤต

ภาคต่อ · ขั้นที่ 5 · แผนหลังเรียนรู้ k = 2.8 นาที (ของจริง 2.8 นาที)

Gantt แผน · meow-shop

แถว = บัญชี · แท่ง = บล็อก (สีตามสกิล) · เวลา = โทเคนประเมิน ÷ ความเร็ว CLI × k · KC-QUEST · MAX_AGENTS=4



รวม ≈ 2.8 นาที · เส้นประ = บล็อกเดียวกันที่ใช้หลายบัญชี

yolo · Claude 1 บัญชี | cyolo · Codex 1 บัญชี | image · Codex สร้างภาพ | review · ผู้ทำ + ผู้ตรวจคนละ CLI | best_of · N ตัวเลือก + ผู้ตัดสิน

▭ ขอบส้ม = สายวิกฤต

สรุป

ไม่มีแผน vs มี dag-planner

	บท 04	บท 05
ต่อยอดจากอะไร	–	base skills ใน ./base_skills
ใครเขียน graph.json	คน / AI เขียนเอง	export จาก blocks.json
แต่ละงานใช้สกิลอะไร	ระบุเองทีละ node	ทุกบล็อกมี skill + difficulty
ใครทำบล็อกไหน เมื่อไร	ไม่รู้จนกว่าจะรัน	Gantt แผน (KC-QUEST + โควตาจริง)
เสร็จเมื่อไร	ไม่รู้	1.7 → จริง 2.8 → แผนใหม่ 2.8 นาที
what-if	ต้องรันจริง	plan ใหม่ ไม่ใช้โทเคนของทีม
การบ้าน	–	10 โจทย์ วางแผนเป็น DAG + Gantt พร้อมเฉลย

ประเด็นสำคัญ

จำ 3 ข้อนี้

1 · บล็อก

ทุกบล็อกบอก **สกิล** ความยาก ไฟล์ผล และเกณฑ์เสร็จ

2 · แผน

DAG + Gantt บอกว่าใครทำอะไร เมื่อไร ก่อนเสียโทเคน

3 · ฐาน

ต่อยอดจาก **base skills** ไม่คัดลอกโค้ด ไม่แก้ของเดิม

ข้อควรรู้

ข้อควรรู้

แผนเป็นค่าประมาณ

แผนแรกต่างจากจริงได้มาก calib.json ช่วยให้แม่นขึ้น

วางแผนได้โดยไม่มีบัญชี

ทุกบัญชีใน --accounts ระบุ % สกิลไม่เรียกบัญชีใดเลย

บัญชีอาจไม่ตรงแผน

export ส่งเป็นฝั่ง ถ้าจะห้ามบัญชีไหนให้ตัดจาก --accounts

ลูกศรที่ขาด

ตัด dependency เฉพาะงานที่ไม่ใช้ข้อมูลของกันจริง

บล็อกหลายบัญชี

best_of n=2 ใช้ 3 บัญชีพร้อมกัน agent สูงสุดต้องไม่ต่ำกว่านั้น

base skills อยู่ข้างสกิล

ใน shared/skills เดียวกัน ห้ามแก้ไฟล์ใน ./base_skills

คำสั่งที่ใช้บ่อย

ใน container

```
D=/root/ai-accounts/.Claude/shared/skills/dag-planner/scripts/dagplan.py
python3 $D doctor                                  # เครื่องพร้อมไหม
python3 $D dag meow-shop --blocks blocks.json      # ตรวจบล็อก + rank + dag.png
python3 $D plan meow-shop --max-agents 4           # Gantt แผนจากโควตาจริง
python3 $D plan meow-shop --accounts yolo1:5,yolo2,cyolo1,cyolo2   # what-if (สมมติโควตา)
python3 $D export meow-shop && python3 $D run meow-shop --detach   # ภาคต่อ: รันจริง (ใช้โทเคน)
python3 $D report meow-shop                        # ภาคต่อ: Gantt จริง + แผน vs จริง + calib.json
```

จาก host

```
docker exec multi-account cat /root/ai-accounts/.Claude/shared/skills/dag-planner/SKILL.md
docker exec -i multi-account python3 /root/ai-accounts/.Claude/shared/skills/dag-planner/scripts/dagplan.py dag meow-shop --blocks - < blocks.json
```

คำสั่งที่ใช้บ่อย · บนเครื่องโดยตรง ไม่ผ่าน container

รันจากโฟลเดอร์ 05_DAG_Planner ใช้ ./base_skills และโควตาสมมติ

```
export DAGPLAN_DIR=./workspace/plans
D=_solution/skills/dag-planner/scripts/dagplan.py
python3 $D dag hw02-gov-rag-k8s --blocks _solution/homework/hw02-gov-rag-k8s/blocks.json
python3 $D plan hw02-gov-rag-k8s --max-agents 4 --accounts yolo1:80,yolo2:60,cyolo1:90,cyolo2:70
```

ทุกบัญชีใน `--accounts` ระบุ % = สมมติทั้งหมด สกิลไม่เรียกบัญชีใดเลย · ต้องมี Python 3.8+ · ไม่มี headless Chromium จะได้ภาพเป็น SVG · เฉลยการบ้านทำแบบนี้

เช็กลิสต์

ทำครบหรือยัง?

- [ ] มีโฟลเดอร์ `skills/dag-planner` ที่สร้างจาก `./base_skills` และ TEST-REPORT ผ่าน (ขั้นที่ 1)
- [ ] base skills 3 ตัวและ dag-planner อยู่ใน shared ทั้งสองฝั่ง ทุกบัญชีเห็นสกิล และ doctor ผ่าน (ขั้นที่ 2)
- [ ] ได้ dag.png และ gantt-plan.png ของเหมียวช็อปที่มีป้ายสกิลทุกบล็อก โดยยังไม่รัน (ขั้นที่ 3)
- [ ] การบ้านครบ 10 แผน แผนละ 5 ไฟล์ และไม่ได้สั่ง export · run · report
- [ ] คิด rank ของบล็อกบนสายวิกฤตด้วยมือแล้วตรงกับภาพ

สรุป

วางแผนก่อน
แล้วค่อยให้ทีมทำตามแผน

บล็อกบอก **สกิล** · DAG บอก **ลำดับ** · Gantt บอก **ใคร เมื่อไร** · base skills เป็น **ฐาน**
ต่อด้วยบท 06 · KC-QUEST เพื่อดูสมการแจกบล็อกให้บัญชีแบบละเอียด

รายละเอียดทุกขั้นตอนและพรอมต์เต็มอยู่ใน README.md